

# Kernel-Mode Side-Channel Detection & Mitigation Driver

A comprehensive Ring 0 driver design for Windows 11 that detects suspicious side-channel attack behavior, compromised kernels, and hardware keyloggers, while actively mitigating cache and memory side-channel attacks targeting sensitive data through PMU monitoring, IDT integrity verification, VERW/L1D flush barriers, and cache partitioning strategies.

**Platform:** Windows 11 x64 (WDM/KMDF)

**Classification:** Ring 0 Kernel Driver

**Version:** 1.0-DRAFT

**Date:** May 2026

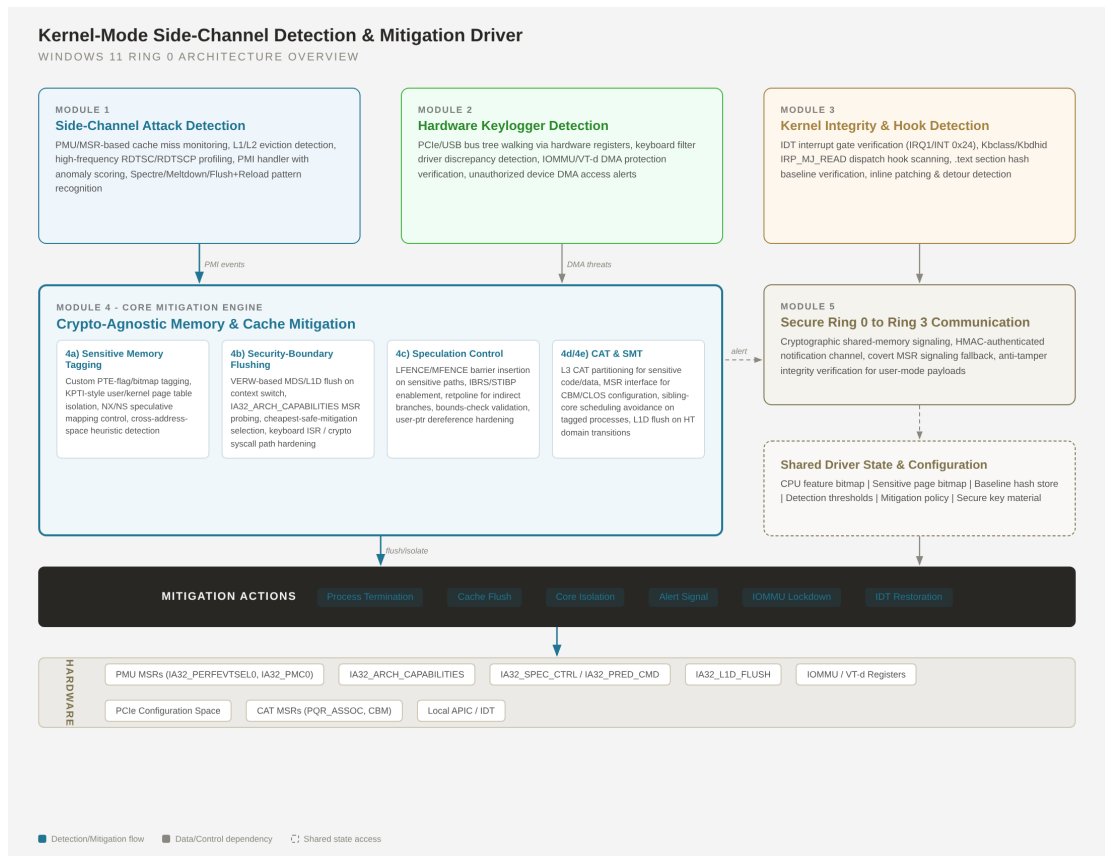
# Table of Contents

|                                                                    |    |
|--------------------------------------------------------------------|----|
| 1. Executive Summary and Architecture Overview                     | 2  |
| 1.1 Key MSR and Hardware Register Reference . . . . .              | 3  |
| 2. Side-Channel Attack Detection (Cache and Timing)                | 4  |
| 2.1 PMU Configuration and Event Selection . . . . .                | 4  |
| 2.2 PMI Handler Implementation . . . . .                           | 5  |
| 2.3 RDTSC/RDTSCP Profiling from Ring 3 . . . . .                   | 6  |
| 3. Hardware Keylogger and Compromised Peripheral Detection         |    |
| 3.1 PCIe Bus Tree Walking via Configuration Space . . . . .        | 8  |
| 3.2 IOMMU/VT-d DMA Protection Verification . . . . .               | 9  |
| 4. Kernel Integrity and Hook Detection                             | 10 |
| 4.1 IDT Integrity Verification . . . . .                           | 10 |
| 4.2 Driver Dispatch Routine Hook Detection . . . . .               | 11 |
| 4.3 Kernel .text Section Integrity Verification . . . . .          | 12 |
| 5. Crypto-Agnostic Memory and Cache Mitigation Module              | 14 |
| 5.1 Sensitive Memory Identification and Isolation . . . . .        | 14 |
| 5.2 Security-Boundary Cache and Buffer Flushing . . . . .          | 15 |
| 5.3 Speculation and Transient Execution Controls . . . . .         | 17 |
| 5.4 Cache Partitioning via CAT (Cache Allocation Technology) . . . | 18 |
| 5.5 Hyper-Threading and Cross-VM Considerations . . . . .          | 19 |
| 6. Secure Ring 0 to Ring 3 Communication                           | 20 |
| 7. Integration and Cross-Module Interactions                       | 21 |
| 7.1 Shared Driver State Structure . . . . .                        | 21 |
| 7.2 Cross-Module Event Flow . . . . .                              | 22 |
| 8. Constraints, IRQL Rules, and Best Practices                     | 23 |
| 8.1 IRQL and Paging Constraints . . . . .                          | 23 |
| 8.2 MSR and CPUID Access Best Practices . . . . .                  | 24 |
| 8.3 Avoiding New Exploitable Gadgets . . . . .                     | 24 |
| 8.4 Performance Impact Assessment . . . . .                        | 25 |

# 1. Executive Summary and Architecture Overview

This document presents the architectural design and critical code paths for a Windows 11 kernel-mode driver (WDM/KMDF) that operates at Ring 0 to detect and actively mitigate side-channel attacks, compromised kernels, and hardware-level keyloggers. The driver is designed under the fundamental assumption that the kernel itself may be partially compromised, meaning it cannot rely entirely on kernel APIs for its own integrity verification. Instead, it employs direct hardware register access via MSRs and CPUID, cross-validation of OS-reported state against hardware ground truth, and cryptographic integrity checking of its own code and critical kernel structures.

The architecture is organized into five synergistic modules that share a common state and configuration layer. Module 1 focuses on side-channel attack detection using CPU Performance Monitoring Units (PMUs), configuring MSRs to monitor cache eviction patterns and high-frequency timestamp counter reads that are indicative of Spectre, Meltdown, Flush+Reload, and similar cache-timing attacks. Module 2 addresses hardware keylogger and compromised peripheral detection by directly walking the PCIe and USB bus hierarchies through hardware registers, bypassing the OS device stack that could be hooked by a rootkit. Module 3 provides kernel integrity verification by checking the Interrupt Descriptor Table (IDT), driver dispatch routines, and the .text sections of loaded modules against secure baselines. Module 4 is the core mitigation engine, implementing crypto-agnostic memory tagging, security-boundary cache flushing, speculation control barriers, and optional cache partitioning strategies. Module 5 provides a secure communication channel from Ring 0 to Ring 3, designed to resist interception even when the kernel may be compromised.



**Figure 1.** High-level driver architecture showing five modules, shared state, mitigation action pipeline, and hardware interface layer.

The design philosophy centers on defense-in-depth and zero-trust of the kernel API surface. Each module validates its inputs independently and cross-references findings with other modules. For example, a PMU anomaly detected by Module 1 may trigger enhanced cache flushing by Module 4, while a hook detected by Module 3 may cause Module 2 to perform deeper DMA protection checks. All modules communicate through a shared state structure that is itself integrity-verified using a hash stored in a TPM-protected location. The driver is designed to be loaded early in the boot sequence (as a boot-start driver) to minimize the window of vulnerability before its protections are active.

## 1.1 Key MSR and Hardware Register Reference

The following table summarizes the primary MSRs and hardware registers used across all modules. These registers are accessed directly through `__readmsr()` intrinsics and `CPUID` leaf queries, avoiding any indirection through potentially-hooked kernel APIs. The selection of registers is based on Intel SDM Volume 4 and the AMD Architecture Programmer's Manual, covering performance monitoring, architectural capabilities, speculation control, and cache allocation technology.

| Register / MSR         | Addresses   | Module | Purpose                                                    |
|------------------------|-------------|--------|------------------------------------------------------------|
| IA32_PERFEVTSEL0       | 0x186       | M1     | Event select for L1D cache miss / eviction events          |
| IA32_PMC0              | 0x0C1       | M1     | Performance counter for selected PMU event                 |
| IA32_PERF_GLOBAL_CTRL  | 0x38F       | M1     | Global enable/disable for PMU counters                     |
| IA32_FIXED_CTR0        | 0x309       | M1     | Fixed-function instr retired counter                       |
| IA32_ARCH_CAPABILITIES | 0x10A       | M4     | RDCL_NO, IBRS_ALL, RSBA, MDS_NO, L1TF_NO flags             |
| IA32_SPEC_CTRL         | 0x48        | M4     | IBRS / STIBP enable control                                |
| IA32_PRED_CMD          | 0x49        | M4     | IBPB command register (indirect branch prediction barrier) |
| IA32_FLUSH_CMD         | 0x10B       | M4     | L1D_FLUSH command register                                 |
| IA32_VIRT_SPEC_CTRL    | 0xC001_011F | M4     | AMD virtual speculation control                            |
| MSR_PQR_ASSOC          | 0xC8F       | M4     | CAT Class of Service (CLOS) association                    |
| MSR_L3_CBMBASE         | 0xC90       | M4     | L3 Cache Bit Mask for CAT capacity bitmask                 |
| VT-d DMAR              | MMIO        | M2     | DMA Remapping Reporting structure for IOMMU validation     |

**Table 1.** Primary MSR and hardware register reference used across all driver modules.

## 2. Side-Channel Attack Detection (Cache and Timing)

This module leverages the CPU's Performance Monitoring Units (PMUs) to detect anomalous cache behavior that is indicative of side-channel attacks. Modern side-channel attacks such as Spectre, Meltdown, Flush+Reload, Evict+Time, and Prime+Probe all share a common characteristic: they produce abnormal patterns of cache misses and evictions that can be statistically distinguished from normal application behavior. By programming the PMU to count L1 data cache misses, L2 cache evictions, and last-level cache (LLC) misses, and by monitoring the rate at which Ring 3 processes execute RDTSC/RDTSCP instructions, this module builds a behavioral profile that triggers alerts when thresholds are exceeded.

### 2.1 PMU Configuration and Event Selection

The PMU is configured by writing to the IA32\_PERFEVTSEL0-3 MSRs (model-specific registers at addresses 0x186-0x189) to select the events to be monitored. Each IA32\_PERFEVTSEL register is a 32-bit field where bits [7:0] specify the event select (umask + event code), bit 12 is the enable flag, bit 16 enables counting in user mode (CPL > 0), bit 17 enables counting in OS mode (CPL = 0), and bit 20 enables the PMI (Performance Monitor Interrupt) on counter overflow. The corresponding counters are read from IA32\_PMC0-3 (addresses 0x0C1-0x0C4). The following code configures the PMU to monitor L1D cache misses and sets up the PMI handler to fire when the counter overflows, which indicates an unusually high rate of cache misses.

```
// ===== // PMU Configuration:
// L1D Cache Miss + RDTSC Profiling // Target: Windows 11 x64, Intel Skylake+ / AMD Zen+ //
// IRQ: Must be called at PASSIVE_LEVEL (driver init) //
===== #include <ntddk.h> #include
<intrin.h> // — MSR Addresses (Intel SDM Vol 4) — #define IA32_PERFEVTSEL0 0x186
#define IA32_PERFEVTSEL1 0x187 #define IA32_PMC0 0x0C1 #define IA32_PMC1 0x0C2 #define
IA32_PERF_GLOBAL_CTRL 0x38F #define IA32_PERF_GLOBAL_OVF_CTRL 0x390 #define
IA32_FIXED_CTR0 0x309 // Instructions retired #define IA32_FIXED_CTR_CTRL 0x38D #define
IA32_APIC_BASE 0x1B // — Event Select Values (Intel Skylake Umask + Event) — #define
EVT_L1D_CACHE_MISS 0x0051 // MEM_LOAD_RETIRED.L1_MISS #define EVT_L2_CACHE_MISS 0x0028
// L2_RQSTS.MISS #define EVT_RDTSC_INSTR 0x003C // CPU_CLK_UNHALTED.RING0_TRANS (proxy)
#define EVT_ITLB_MISS 0x0085 // ITLB_MISSES.WALK_COMPLETED // — Thresholds (tunable via
registry) — #define L1D_MISS_THRESHOLD 50000 // Per-sample-window #define
L2_MISS_THRESHOLD 20000 #define RDTSC_RATE_THRESHOLD 100000 // RDTSC/second from Ring 3
// — PMI Context (non-paged, accessed at HIGH_LEVEL) — typedef struct _PMU_CONTEXT {
volatile LONG L1MissCount; volatile LONG L2MissCount; volatile LONG RdtscRate; volatile
LONG AlertLevel; // 0=normal, 1=watch, 2=critical ULONG SampleWindow; // milliseconds
between resets ULONG CpuFeatures; // CPU feature bitmap PKINTERRUPT PmiInterrupt; // PMI
interrupt object ULONG PmiVector; // Allocated vector } PMU_CONTEXT, *PPMU_CONTEXT;
PMU_CONTEXT g_PmuCtx[MAXIMUM_PROCESSORS]; // — Configure IA32_PERFEVTSEL0 for L1D Cache
Miss — NTSTATUS PmuConfigureL1DMiss(ULONG CpuIndex) { PPMU_CONTEXT ctx =
&g_PmuCtx[CpuIndex]; // Step 1: Disable counter before reconfiguring
__writemsr(IA32_PERFEVTSEL0, 0); __writemsr(IA32_PMC0, 0); // Step 2: Write event select
// Bits [7:0] = Event Select (0x51) // Bits [15:8] = Umask (0x01 for L1_MISS) // Bit 12 =
Enable counter // Bit 16 = USR (count in Ring 3) // Bit 17 = OS (count in Ring 0) // Bit
20 = PMI on overflow ULONG64 evtSel = (1ULL << 16) | // USR mode (1ULL << 17) | // OS
mode (1ULL << 20) | // PMI on overflow (1ULL << 12) | // Enable ((UINT64)0x01 << 8) | //
Umask = L1_MISS 0x51; // Event = MEM_LOAD_RETIRED __writemsr(IA32_PERFEVTSEL0, evtSel);
// Step 3: Pre-load counter near overflow for early PMI // PMC is 48-bit on Skylake; set
to (2^48 - threshold) // so PMI fires after 'threshold' events ULONG64 counterMax = (1ULL
<< 48) - 1; ULONG64 initialValue = counterMax - L1D_MISS_THRESHOLD;
__writemsr(IA32_PMC0, initialValue); // Step 4: Enable global PMU ULONG64 globalCtrl =
__readmsr(IA32_PERF_GLOBAL_CTRL); globalCtrl |= (1ULL << 0); // Enable PMC0
__writemsr(IA32_PERF_GLOBAL_CTRL, globalCtrl); ctx->L1MissCount = 0; return
STATUS_SUCCESS; } // — Configure IA32_PERFEVTSEL1 for L2 Cache Miss — NTSTATUS
PmuConfigureL2Miss(ULONG CpuIndex) { PPMU_CONTEXT ctx = &g_PmuCtx[CpuIndex];
__writemsr(IA32_PERFEVTSEL1, 0); __writemsr(IA32_PMC1, 0); ULONG64 evtSel = (1ULL << 16)
| // USR (1ULL << 17) | // OS (1ULL << 20) | // PMI on overflow (1ULL << 12) | // Enable
((UINT64)0x3F << 8) | // Umask: all L2 miss sources 0x24; // Event: L2_RQSTS.MISS
__writemsr(IA32_PERFEVTSEL1, evtSel); ULONG64 counterMax = (1ULL << 48) - 1;
__writemsr(IA32_PMC1, counterMax - L2_MISS_THRESHOLD); ULONG64 globalCtrl =
__readmsr(IA32_PERF_GLOBAL_CTRL); globalCtrl |= (1ULL << 1); // Enable PMC1
__writemsr(IA32_PERF_GLOBAL_CTRL, globalCtrl); ctx->L2MissCount = 0; return
STATUS_SUCCESS; }
```

## 2.2 PMI Handler Implementation

The Performance Monitor Interrupt (PMI) handler is the critical path that executes when a PMU counter overflows. It must execute at HIGH\_LEVEL (IRQL = 26 on x64, corresponding to the local APIC performance counter vector) and must not access any paged memory. The handler reads the overflow status from IA32\_PERF\_GLOBAL\_STATUS, increments the appropriate anomaly counters, evaluates the alert level based on running thresholds, and if the alert level reaches "critical," it can trigger immediate mitigation actions such as flushing the L1D cache or isolating the current logical processor. The handler also re-arms the counter by writing to IA32\_PERF\_GLOBAL\_OVF\_CTRL to clear the overflow condition and re-loading the PMC with the threshold-relative initial value.

```
// ===== // PMI Handler: Called
// at HIGH_LEVEL (IRQL 26) // CRITICAL: No paged memory access. No kernel API calls. //
// ===== // Forward declaration VOID
PmiIsr(PK_INTERRUPT Interrupt, PVOID Context); // — Register PMI with the local APIC —
NTSTATUS PmuRegisterPmi(ULONG CpuIndex) { PPMU_CONTEXT ctx = &g_PmuCtx[CpuIndex];
IO_INTERRUPT_VECTORINFO vectorInfo = {0}; // Allocate a vector from the IDT for PMI // On
x64 Windows, we use KeRegisterInterruptHandler // to hook the APIC performance counter
vector (0xE4 typically) ULONG vector = 0xE4; // Local APIC LVT Perf Counter vector // Map
the APIC LVT Performance Counter entry // IA32_APIC_BASE + 0x340 = LVT Perf Counter
Register // Bits [7:0] = vector, Bit 16 = Mask, Bits [10:8] = Delivery mode
PHYSICAL_ADDRESS apicBase; apicBase.QuadPart = (__readmsr(IA32_APIC_BASE) & ~0xFFF) +
0x340; PULONG lvtPerf = (PULONG)MmMapIoSpace(apicBase, 4, MmNonCached); if (!lvtPerf)
return STATUS_INSUFFICIENT_RESOURCES; // Unmask and set vector, delivery mode = fixed
(0) *lvtPerf = (vector & 0xFF); // Unmasked, fixed delivery MmUnmapIoSpace(lvtPerf, 4);
ctx->PmiVector = vector; // Register our ISR for this vector NTSTATUS status =
IoConnectInterrupt(&ctx->PmiInterrupt, PmiIsr, // ISR routine (PVOID)ctx, // Context =
our PMU_CONTEXT NULL, // SpinLock (not needed for PMI) vector, // Vector HIGH_LEVEL, //
IRQL HIGH_LEVEL, // Synchronize IRQL LevelSensitive, // PMI is level-triggered FALSE, //
Not shareable 0, // Processor affinity (set per-CPU) FALSE // Floating save ); return
status; } // — PMI ISR (runs at HIGH_LEVEL) — VOID PmiIsr(PK_INTERRUPT Interrupt, PVOID
Context) { PPMU_CONTEXT ctx = (PPMU_CONTEXT)Context; // 1. Read global overflow status
ULONG64 globalStatus = __readmsr(IA32_PERF_GLOBAL_STATUS); // 2. Check which counter
overflowed if (globalStatus & (1ULL << 0)) { // PMC0 overflowed = L1D miss
InterlockedIncrement(&ctx->L1DMissCount); // Re-arm PMC0: clear overflow, reload counter
__writemsr(IA32_PERF_GLOBAL_OVF_CTRL, (1ULL << 0)); ULONG64 counterMax = (1ULL << 48) -
1; __writemsr(IA32_PMC0, counterMax - L1D_MISS_THRESHOLD); } if (globalStatus & (1ULL <<
1)) { // PMC1 overflowed = L2 miss InterlockedIncrement(&ctx->L2MissCount);
__writemsr(IA32_PERF_GLOBAL_OVF_CTRL, (1ULL << 1)); ULONG64 counterMax = (1ULL << 48) -
1; __writemsr(IA32_PMC1, counterMax - L2_MISS_THRESHOLD); } // 3. Evaluate alert level
(lock-free, approximate) LONG l1d = ctx->L1DMissCount; LONG l2 = ctx->L2MissCount; if
(l1d > 10 || l2 > 10) { // CRITICAL: sustained high cache miss rate // This pattern is
strongly indicative of // Flush+Reload or Prime+Probe attacks
InterlockedExchange(&ctx->AlertLevel, 2); // Immediate mitigation: flush L1D cache //
VERW instruction with MDS-clearing microcode __mm_mfence(); // On CPUs with
IA32_FLUSH_CMD support: if (ctx->CpuFeatures & CPU_FEAT_L1D_FLUSH) { __writemsr(0x10B,
1ULL); // IA32_FLUSH_CMD.L1D_FLUSH } } else if (l1d > 3 || l2 > 3) {
InterlockedExchange(&ctx->AlertLevel, 1); // WATCH } // 4. Acknowledge interrupt to
local APIC // Write EOI (End Of Interrupt) to APIC PHYSICAL_ADDRESS apicEoi;
apicEoi.QuadPart = (__readmsr(IA32_APIC_BASE) & ~0xFFF) + 0x0B0; PULONG eoi =
(PULONG)MmMapIoSpace(apicEoi, 4, MmNonCached); if (eoi) { *eoi = 0; MmUnmapIoSpace(eoi,
4); } }
```

## 2.3 RDTSC/RDTSCP Profiling from Ring 3

High-frequency invocation of RDTSC or RDTSCP by unprivileged (Ring 3) processes is a strong indicator of timing-based side-channel reconnaissance. Legitimate applications rarely need sub-microsecond timing resolution, and processes that read the TSC thousands of times per second are typically performing cache eviction timing, fence measurement, or other attack-related activities. This module tracks RDTSC usage by configuring the PMU to count CPU\_CLK\_UNHALTED.RING0\_TRANS events (which count transitions into Ring 0, often caused by RDTSC syscall emulation when TSD is set) or, on CPUs that support it, the OUT\_OF\_ORDER\_CORE\_CYCLES event as a proxy for high-frequency timing loop detection. Additionally, the driver can set the TSD (Time Stamp Disable) bit in CR4 to force RDTSC execution from Ring 3 to trigger a general protection fault (#GP), which the driver's IDT handler can intercept and count, providing a precise per-process RDTSC usage metric.

```
// — RDTSC Profiling via TSD + #GP interception — // When CR4.TSD = 1, RDTSC from Ring
3 causes #GP. // We hook INT 0x0D (#GP) to count per-process RDTSC attempts. typedef
struct _RDTSC_PROFILE { HANDLE ProcessId; ULONG RdtscCount; ULONG LastCountTimestamp; //
Tick count BOOLEAN IsFlagged; } RDTSC_PROFILE, *PRDTSC_PROFILE; #define
MAX_PROFILED_PROCS 256 RDTSC_PROFILE g_RdtscProfiles[MAX_PROFILED_PROCS]; KSPIN_LOCK
g_RdtscLock; // — Enable TSD bit in CR4 — VOID EnableTSD(VOID) { ULONG64 cr4 =
__readcr4(); cr4 |= (1ULL << 2); // Set TSD bit (bit 2) __writecr4(cr4); } // — #GP
Handler wrapper for RDTSC counting — // This is installed as a custom IDT entry for INT
0x0D // The original #GP handler is saved and called after counting ULONG64
g_OriginalGpHandler = 0; VOID RdtscGpHandler( PKTRAP_FRAME TrapFrame, ULONG64 ErrorCode)
{ UNREFERENCED_PARAMETER(ErrorCode); // Only count Ring 3 #GP faults (CPL from CS
segment) USHORT cs = (USHORT)(TrapFrame->SegCs & 0x3); if (cs == 3) { // Ring 3 // Check
if faulting instruction is RDTSC (0x0F 0x31) PCHAR rip = (PCHAR)TrapFrame->Rip; __try {
if (rip[0] == 0x0F && rip[1] == 0x31) { // RDTSC from Ring 3 detected HANDLE pid =
PsGetCurrentProcessId(); KIRQL oldIrql; KeAcquireSpinLock(&g_RdtscLock, &oldIrql); for
(int i = 0; i < MAX_PROFILED_PROCS; i++) { if (g_RdtscProfiles[i].ProcessId == pid) {
g_RdtscProfiles[i].RdtscCount++; ULONG now = KeQueryTimeIncrement(); if
(g_RdtscProfiles[i].RdtscCount > RDTSC_RATE_THRESHOLD) { g_RdtscProfiles[i].IsFlagged =
TRUE; } break; } if (g_RdtscProfiles[i].ProcessId == NULL) {
g_RdtscProfiles[i].ProcessId = pid; g_RdtscProfiles[i].RdtscCount = 1; break; } }
KeReleaseSpinLock(&g_RdtscLock, oldIrql); // Emulate RDTSC: read TSC and store in
RDX:RAX // then advance RIP past the 2-byte instruction ULONG64 tsc = __rdtsc();
TrapFrame->Rdx = (tsc >> 32) & 0xFFFFFFFF; TrapFrame->Rax = tsc & 0xFFFFFFFF;
TrapFrame->Rip += 2; // Skip RDTSC instruction return; // Handled, don't chain to
original } } __except(EXCEPTION_EXECUTE_HANDLER) { // RIP was invalid, fall through to
original handler } } // Chain to original #GP handler for non-RDTSC faults // (This is
architecture-specific; on x64 Windows the // original handler address is in the saved IDT
entry) }
```

### 3. Hardware Keylogger and Compromised Peripheral Detection

This module detects hardware-based keyloggers and compromised peripherals by directly walking the PCIe and USB bus hierarchies through hardware configuration space registers, bypassing the OS device stack which could be subverted by a rootkit. A kernel-level keylogger typically operates either as an

unauthorized filter driver in the keyboard device stack or as a DMA-capable malicious device that reads keystroke buffers directly from system memory. The detection strategy addresses both vectors: first, by comparing the hardware devices reported by the bus controller against the OS's claimed keyboard and filter drivers to detect discrepancies; and second, by reading the IOMMU/VT-d DMA remapping registers to verify that no unauthorized devices have been granted DMA access to memory regions containing keystroke data.

### 3.1 PCIe Bus Tree Walking via Configuration Space

PCIe devices are enumerated by reading the PCIe Enhanced Configuration Access Mechanism (ECAM) space, which maps the entire PCIe configuration space into a 256-MB MMIO region starting at the base address found in the MCFG ACPI table. Each device's configuration space is 4 KB, containing the standard PCI header (Vendor ID, Device ID, Class Code, BAR registers, etc.) at offset 0x00-0x3F. By reading these registers directly from MMIO, we bypass any kernel-level hooking of the IRP\_MN\_QUERY\_DEVICE\_RELATIONS handler or the PCI bus driver's enumeration routines. The following code walks the PCIe bus tree, collects device information, and compares it against the OS-reported device list to detect discrepancies.

```
// ===== // PCIe Bus Walking via
// ECAM (Enhanced Configuration Access) // Bypasses OS device stack – reads hardware
// registers directly // IRQL: PASSIVE_LEVEL (uses MmMapIoSpace) //
===== #pragma pack(push, 1)
typedef struct _PCI_CONFIG_HEADER { USHORT VendorId; // Offset 0x00 USHORT DeviceId; //
Offset 0x02 USHORT Command; // Offset 0x04 USHORT Status; // Offset 0x06 UCHAR
RevisionId; // Offset 0x08 UCHAR ClassCode[3]; // Offset 0x09 (ProgIF, SubClass,
BaseClass) UCHAR CacheLineSize; // Offset 0x0C UCHAR LatencyTimer; // Offset 0x0D UCHAR
HeaderType; // Offset 0x0E UCHAR Bist; // Offset 0x0F ULONG Bar[6]; // Offset 0x10-0x27
ULONG CardbusCis; // Offset 0x28 USHORT SubsystemVendorId; // Offset 0x2C USHORT
SubsystemDeviceId; // Offset 0x2E ULONG ExpansionRom; // Offset 0x30 UCHAR
CapabilitiesPtr; // Offset 0x34 UCHAR Reserved[7]; // Offset 0x35-0x3B UCHAR
InterruptLine; // Offset 0x3C UCHAR InterruptPin; // Offset 0x3D UCHAR MinGnt; // Offset
0x3E UCHAR MaxLat; // Offset 0x3F } PCI_CONFIG_HEADER, *PPCI_CONFIG_HEADER; #pragma
pack(pop) // — Discovered device entry — typedef struct _PCI_DISCOVERY_ENTRY { UCHAR
Bus, Device, Function; USHORT VendorId, DeviceId; UCHAR BaseClass, SubClass; BOOLEAN
IsKeyboardRelated; // HID class or input device BOOLEAN DmaCapable; // Has BAR with DMA
capability ULONG BarSizes[6]; } PCI_DISCOVERY_ENTRY, *PPCI_DISCOVERY_ENTRY; #define
MAX_PCI_DEVICES 512 PCI_DISCOVERY_ENTRY g_PciDevices[MAX_PCI_DEVICES]; ULONG
g_PciDeviceCount = 0; // — ECAM base address from MCFG ACPI table — PHYSICAL_ADDRESS
g_EcamBase = {0}; // — Read PCIe config via ECAM — NTSTATUS PciEcamRead(CHAR Bus,
CHAR Device, CHAR Function, ULONG Offset, PVOID Buffer, ULONG Length) { if
(!g_EcamBase.QuadPart) return STATUS_DEVICE_NOT_READY; // ECAM address: Base + (Bus <<
20 | Device << 15 | Function << 12 | Offset) ULONG64 ecamOffset = ((ULONG64)Bus << 20) |
((ULONG64)Device << 15) | ((ULONG64)Function << 12) | (Offset & 0xFFF); PHYSICAL_ADDRESS
pa; pa.QuadPart = g_EcamBase.QuadPart + ecamOffset; PVOID mapped = MmMapIoSpace(pa,
Length, MmNonCached); if (!mapped) return STATUS_INSUFFICIENT_RESOURCES;
RtlCopyMemory(Buffer, mapped, Length); MmUnmapIoSpace(mapped, Length); return
STATUS_SUCCESS; } // — Walk entire PCIe bus tree — NTSTATUS PciWalkBusTree(VOID) {
g_PciDeviceCount = 0; // Parse MCFG from ACPI to get ECAM base // (Simplified: in
production, walk ACPI tables via // HalGetAcpiTable or direct firmware table lookup) //
g_EcamBase already populated during DriverEntry for (CHAR bus = 0; bus < 256; bus++) {
for (CHAR dev = 0; dev < 32; dev++) { for (CHAR func = 0; func < 8; func++) {
PCI_CONFIG_HEADER hdr = {0}; NTSTATUS status = PciEcamRead( bus, dev, func, 0, &hdr,
```

```

sizeof(hdr)); if (!NT_SUCCESS(status)) continue; if (hdr.VendorId == 0xFFFF) continue;
// No device PPCI_DISCOVERY_ENTRY entry = &g_PciDevices[g_PciDeviceCount]; entry->Bus =
bus; entry->Device = dev; entry->Function = func; entry->VendorId = hdr.VendorId;
entry->DeviceId = hdr.DeviceId; entry->BaseClass = hdr.ClassCode[2]; entry->SubClass =
hdr.ClassCode[1]; // Detect keyboard/input related devices // Class 0x09 = Input device
controller // HID class 0x03 in USB context entry->IsKeyboardRelated = (hdr.ClassCode[2]
== 0x09) || (hdr.ClassCode[2] == 0x0C && hdr.ClassCode[1] == 0x03); // USB HC // Check
DMA capability via BAR entry->DmaCapable = FALSE; for (int b = 0; b < 6; b++) { if
(hdr.Bar[b] != 0) { entry->DmaCapable = TRUE; // Size BAR by writing 0xFFFFFFFF //
(production code would do this carefully) } } g_PciDeviceCount++; if (g_PciDeviceCount
>= MAX_PCI_DEVICES) return STATUS_SUCCESS; // If not multi-function, skip remaining
functions if (func == 0 && !(hdr.HeaderType & 0x80)) break; } } } return STATUS_SUCCESS;
} // — Compare hardware devices against OS device list — NTSTATUS
PciDetectDiscrepancies(VOID) { // Get OS-reported device list via
IoGetDeviceObjectPointer // for \Device\KeyboardClass0, etc. // Then cross-reference
against g_PciDevices[] // Step 1: Enumerate OS keyboard devices UNICODE_STRING kbdName;
RtlInitUnicodeString(&kbdName, L"\\Device\\KeyboardClass0"); for (ULONG idx = 0; idx < 10;
idx++) { WCHAR nameBuf[64]; swprintf_s(nameBuf, 64, L"\\Device\\KeyboardClass%d", idx);
RtlInitUnicodeString(&kbdName, nameBuf); PFILE_OBJECT fileObj = NULL; PDEVICE_OBJECT
devObj = NULL; NTSTATUS status = IoGetDeviceObjectPointer( &kbdName, FILE_READ_DATA,
&fileObj, &devObj); if (!NT_SUCCESS(status)) break; // Walk device stack to find filter
drivers PDEVICE_OBJECT attached = devObj; ULONG filterCount = 0; while
(attached->AttachedDevice) { filterCount++; // Check if this filter driver is in our
whitelist // by comparing driver image name against known list PDRIVER_OBJECT drv =
attached->DriverObject; if (drv && drv->DriverName.Buffer) { // If driver name is NOT in
whitelist, // flag as potential keylogger filter if
(!IsWhitelistedDriver(&drv->DriverName)) { LogAlert(ALERT_UNAUTHORIZED_KBD_FILTER,
"Unauthorized keyboard filter driver: %wZ", &drv->DriverName); } } attached =
attached->AttachedDevice; } if (fileObj) ObDereferenceObject(fileObj); } return
STATUS_SUCCESS; }

```

## 3.2 IOMMU/VT-d DMA Protection Verification

Intel VT-d (Virtualization Technology for Directed I/O) provides DMA remapping, which controls which devices can perform DMA to which physical memory regions. A hardware keylogger that uses DMA to read keystroke buffers must have a valid DMA remapping entry in the IOMMU's page tables. This module reads the VT-d DMAR (DMA Remapping Reporting) structure from the ACPI tables, then walks the VT-d context entries and second-level page tables to verify that no device outside the expected set has DMA access to memory regions where keystroke data is stored (typically, the keyboard class driver's ring buffer in non-paged pool). The verification involves reading the Root-Entry and Context-Entry tables directly from their physical addresses via MmMapIoSpace, bypassing any potential kernel API hooking. If an unauthorized mapping is found, the driver can immediately invalidate the affected context entry by clearing the Present bit in the Context Entry, effectively blocking the malicious device's DMA access.

```

// ===== // IOMMU/VT-d DMA
Protection Verification // Reads VT-d DMAR tables directly from physical memory // IRQL:
PASSIVE_LEVEL // ===== // — VT-d
Register Offsets (MMIO) — #define VTD_VER_REG 0x00 // Version #define VTD_GCMD_REG 0x18
// Global Command #define VTD_GSTS_REG 0x1C // Global Status #define VTD_RTADDR_REG 0x20
// Root Table Address // — Root Entry (16 bytes) — typedef struct _VTD_ROOT_ENTRY {
ULONG64 Present : 1; ULONG64 Reserved : 11; ULONG64 ContextTablePtr : 52; // Physical
address ULONG64 Reserved2; } VTD_ROOT_ENTRY, *PVTD_ROOT_ENTRY; // — Context Entry (16

```

```

bytes) — typedef struct _VTD_CONTEXT_ENTRY { ULONG64 Present : 1; ULONG64 FPD : 1; //
Fault Processing Disable ULONG64 Reserved : 6; ULONG64 SecondLevelPageTranslationPtr :
52; ULONG64 DomainIdentifier : 16; ULONG64 Reserved2 : 48; } VTD_CONTEXT_ENTRY,
*PVTD_CONTEXT_ENTRY; // — Verify DMA remapping for all PCI devices — NTSTATUS
VtdVerifyDmaProtection(VOID) { PHYSICAL_ADDRESS dmarBase = {0}; // Locate DMAR table in
ACPI (production: use firmware API) // dmarBase = AcpiFindTable('DMAR'); // Read VT-d
MMIO base from DMAR structure // For each DMA Remapping Hardware Unit (DRHD):
PHYSICAL_ADDRESS vtdMmio = {0}; // vtdMmio obtained from DRHD Register Base Address PVOID
vtdRegs = MmMapIoSpace(vtdMmio, 0x1000, MmNonCached); if (!vtdRegs) return
STATUS_DEVICE_NOT_READY; // Read Root Table Address ULONG64 rtAddr =
*(PULONG64)((PUCHAR)vtdRegs + VTD_RTADDR_REG); rtAddr &= ~0xFFF; // Clear flags, get
physical address // Map Root Entry Table (256 entries, one per Bus) PVOID rootTable =
MmMapIoSpace( (PHYSICAL_ADDRESS){.QuadPart = (LONGLONG)rtAddr}, 256 *
sizeof(VTD_ROOT_ENTRY), MmNonCached); if (!rootTable) { MmUnmapIoSpace(vtdRegs, 0x1000);
return STATUS_DEVICE_NOT_READY; } // Walk root entries for (ULONG bus = 0; bus < 256;
bus++) { PVTD_ROOT_ENTRY rootEntry = (PVTD_ROOT_ENTRY) ((PUCHAR)rootTable + bus *
sizeof(VTD_ROOT_ENTRY)); if (!rootEntry->Present) continue; // Map Context Table for
this bus PHYSICAL_ADDRESS ctxPhys = {0}; ctxPhys.QuadPart = rootEntry->ContextTablePtr &
~0xFFF; PVOID ctxTable = MmMapIoSpace(ctxPhys, 256 * sizeof(VTD_CONTEXT_ENTRY),
MmNonCached); if (!ctxTable) continue; // Walk context entries (one per Device:Function)
for (ULONG devFunc = 0; devFunc < 256; devFunc++) { PVTD_CONTEXT_ENTRY ctxEntry =
(PVTD_CONTEXT_ENTRY) ((PUCHAR)ctxTable + devFunc * sizeof(VTD_CONTEXT_ENTRY)); if
(!ctxEntry->Present) continue; UCHAR dev = (UCHAR)(devFunc >> 3); UCHAR func =
(UCHAR)(devFunc & 0x7); // Check if this device is in our authorized DMA list // If NOT
authorized but has a context entry, // it may be a DMA-based keylogger if
(!IsAuthorizedDmaDevice(bus, dev, func)) { LogAlert(ALERT_UNAUTHORIZED_DMA,
"Unauthorized DMA mapping: Bus=%d Dev=%d Func=%d " "Domain=%d SLPTPtr=0x%llx", bus, dev,
func, (ULONG)ctxEntry->DomainIdentifier, ctxEntry->SecondLevelPageTranslationPtr); //
MITIGATION: Clear Present bit to block DMA // This is a destructive action — use with
caution // ctxEntry->Present = 0; // Flush IOTLB via VT-d IOTLB Invalidate register } }
MmUnmapIoSpace(ctxTable, 256 * sizeof(VTD_CONTEXT_ENTRY)); } MmUnmapIoSpace(rootTable,
256 * sizeof(VTD_ROOT_ENTRY)); MmUnmapIoSpace(vtdRegs, 0x1000); return STATUS_SUCCESS; }

```

## 4. Kernel Integrity and Hook Detection

This module provides continuous kernel integrity verification by checking critical kernel data structures and code against secure baselines. The primary targets are the Interrupt Descriptor Table (IDT), which must point to legitimate OS interrupt handlers; the keyboard class driver's dispatch routines, which are common targets for inline hooks and detours; and the .text sections of the kernel and loaded drivers, which must match cryptographic hashes stored in a TPM-protected baseline. The module is designed to operate even when the kernel is partially compromised, using direct hardware access and self-verification to detect tampering with its own code.

### 4.1 IDT Integrity Verification

The Interrupt Descriptor Table (IDT) on x64 Windows contains 256 16-byte entries, each describing an interrupt or exception handler. The keyboard controller interrupt (IRQ 1) is mapped to a specific IDT vector, which on systems with IOAPIC is typically INT 0x24 (decimal 36) or 0x31 (decimal 49), depending on the IOAPIC redirection entry configuration. A rootkit that wishes to intercept

keystrokes will often hook this IDT entry, replacing the handler address with a pointer to its own code that logs keystrokes before chaining to the original handler. This module reads the IDT base address from the IDTR register using the SIDT instruction, then reads each entry's handler address and compares it against the expected value derived from the kernel image base and known offsets. Any discrepancy triggers an immediate alert.

```
// ===== // IDT Integrity
Verification // Reads IDT via SIDT, validates keyboard interrupt handler // IRQL:
DISPATCH_LEVEL (no paged access) //
===== // — x64 IDT Entry
Structure (16 bytes) — #pragma pack(push, 1) typedef struct _IDT_ENTRY_64 { USHORT
OffsetLow; // Bits [15:0] of handler offset USHORT Selector; // Code segment selector
UCHAR IstIndex : 3; // IST index UCHAR Reserved1 : 5; UCHAR Type : 4; // Gate type (0xE =
interrupt gate) UCHAR Dpl : 2; // Descriptor privilege level UCHAR Present : 1; UCHAR
Reserved2 : 1; USHORT OffsetMid; // Bits [31:16] of handler offset ULONG OffsetHigh; //
Bits [63:32] of handler offset ULONG Reserved3; } IDT_ENTRY_64, *PIDT_ENTRY_64; #pragma
pack(pop) // — IDTR Structure (10 bytes) — #pragma pack(push, 1) typedef struct _IDTR {
USHORT Limit; ULONG64 Base; } IDTR, *PIDTR; #pragma pack(pop) // — Expected keyboard ISR
handler range — // On Windows 11, the keyboard ISR is typically // KiIsrThunk+X within
ntoskrnl.exe, or // HalpInterruptControllerDispatch for IOAPIC ULONG64
g_ExpectedKbdIsrStart = 0; ULONG64 g_ExpectedKbdIsrEnd = 0; // — Verify IDT entry for
keyboard interrupt — NTSTATUS IdtVerifyKeyboardHandler(ULONG CpuIndex) { IDTR idtr; //
SIDT stores the IDTR contents at the given memory address // This intrinsic is safe and
cannot be hooked __sidt(&idtr); // Map IDT entries PIDT_ENTRY_64 idtBase =
(PIDT_ENTRY_64)idtr.Base; // On IOAPIC systems, keyboard IRQ1 is typically // mapped to
vector 0x24 (36) or 0x31 (49) // We check both common vectors ULONG kbdVectors[] = {
0x24, 0x31, 0x21 }; ULONG numVectors = sizeof(kbdVectors) / sizeof(ULONG); for (ULONG v =
0; v < numVectors; v++) { PIDT_ENTRY_64 entry = &idtBase[kbdVectors[v]]; // Reconstruct
64-bit handler address ULONG64 handler = ((ULONG64)entry->OffsetHigh << 32) |
((ULONG64)entry->OffsetMid << 16) | ((ULONG64)entry->OffsetLow); // Check gate type:
must be interrupt gate (0xE) if (entry->Type != 0xE) { LogAlert(ALERT_IDT_HOOK, "CPU%d:
IDT[0x%02X] gate type invalid: 0x%X " "(expected 0xE). Handler=0x%llX", CpuIndex,
kbdVectors[v], entry->Type, handler); continue; } // Check DPL: must be 0 (Ring 0 only)
if (entry->Dpl != 0) { LogAlert(ALERT_IDT_HOOK, "CPU%d: IDT[0x%02X] DPL=%d (expected 0).
" "Handler=0x%llX", CpuIndex, kbdVectors[v], entry->Dpl, handler); } // Check if handler
falls within ntoskrnl or HAL ULONG64 ntBase = GetNtoskrnlBase(); ULONG64 ntEnd = ntBase +
GetNtoskrnlSize(); if (handler < ntBase || handler >= ntEnd) { // Handler is OUTSIDE
ntoskrnl — likely hooked LogAlert(ALERT_IDT_HOOK, "CPU%d: IDT[0x%02X] handler 0x%llX is
outside " "ntoskrnl [0x%llX - 0x%llX]. POSSIBLE ROOTKIT.", CpuIndex, kbdVectors[v],
handler, ntBase, ntEnd); } // Verify selector is kernel code segment (0x08 or 0x10) if
(entry->Selector != 0x08 && entry->Selector != 0x10) { LogAlert(ALERT_IDT_HOOK, "CPU%d:
IDT[0x%02X] selector=0x%X unexpected.", CpuIndex, kbdVectors[v], entry->Selector); } }
return STATUS_SUCCESS; }
```

## 4.2 Driver Dispatch Routine Hook Detection

On Windows, keyboard data flows from the hardware through a layered driver stack: the port driver (i8042prt or Kbdhid), the class driver (Kbclass), and any optional filter drivers. A kernel keylogger typically installs itself as an upper or lower filter driver in this stack, intercepting IRP\_MJ\_READ requests that carry keystroke data. Alternatively, it may patch the dispatch function pointer in the DRIVER\_OBJECT's MajorFunction array, or insert an inline hook (detour) at the start of the dispatch function body by overwriting the first bytes with a JMP instruction. This module detects both techniques by reading the DRIVER\_OBJECT

directly from the kernel's object manager and inspecting the first bytes of each dispatch routine for common hook prologues.

```
// ===== // Driver Dispatch Hook
Detection (Kbclass / Kbdhid) // Checks IRP_MJ_READ dispatch for inline hooks // IRQL:
PASSIVE_LEVEL // ===== // —
Common hook prologue signatures — typedef struct _HOOK_SIGNATURE { UCHAR Pattern[8];
UCHAR Mask[8]; // 1=must match, 0=don't care CHAR Description[64]; } HOOK_SIGNATURE,
*PHOOK_SIGNATURE; HOOK_SIGNATURE g_HookSignatures[] = { // JMP rel32 (E9 xx xx xx xx) {
{0xE9, 0,0,0,0, 0,0,0,0}, {0xFF, 0,0,0,0, 0,0,0,0}, "JMP rel32 (direct detour)" }, // PUSH
addr; RET (68 xx xx xx xx C3) { {0x68, 0,0,0,0, 0xC3, 0,0}, {0xFF, 0,0,0,0, 0xFF, 0,0},
"PUSH+RET (absolute detour)" }, // JMP [rip+disp32] (FF 25 xx xx xx xx) { {0xFF, 0x25,
0,0,0,0, 0,0}, {0xFF, 0xFF, 0,0,0,0, 0,0}, "JMP [RIP+disp] (indirect detour)" }, // MOV
RAX, imm64; JMP RAX (48 B8 xx 7x FF E0) { {0x48, 0xB8, 0,0,0,0, 0,0}, {0xFF, 0xFF,
0,0,0,0, 0,0}, "MOV RAX, imm64 (possible detour)" }, }; ULONG g_NumHookSigs =
sizeof(g_HookSignatures) / sizeof(HOOK_SIGNATURE); // — Check dispatch routine for
hooks — NTSTATUS CheckDispatchHook( PDRIVER_OBJECT DriverObj, UCHAR MajorFunction,
PWSTR DriverName) { if (!DriverObj) return STATUS_INVALID_PARAMETER; ULONG64
dispatchAddr = (ULONG64)DriverObj->MajorFunction[MajorFunction]; if (!dispatchAddr)
return STATUS_SUCCESS; // Read first 8 bytes of dispatch function UCHAR prologue[8];
__try { RtlCopyMemory(prologue, (PVOID)dispatchAddr, 8); }
__except(EXCEPTION_EXECUTE_HANDLER) { return STATUS_ACCESS_VIOLATION; } // Check against
known hook signatures for (ULONG i = 0; i < g_NumHookSigs; i++) { BOOLEAN match = TRUE;
for (ULONG j = 0; j < 8; j++) { if (g_HookSignatures[i].Mask[j] && prologue[j] !=
g_HookSignatures[i].Pattern[j]) { match = FALSE; break; } } if (match) {
LogAlert(ALERT_DISPATCH_HOOK, "HOOK DETECTED: %ws MajorFunction[%d] at 0x%llx " "matches
signature: %s", DriverName, MajorFunction, dispatchAddr,
g_HookSignatures[i].Description); return STATUS_DEVICE_DATA_ERROR; } } // Additional
check: verify dispatch address is within // the driver's image section ULONG64 drvStart =
(ULONG64)DriverObj->DriverStart; ULONG64 drvEnd = drvStart + DriverObj->DriverSize; if
(dispatchAddr < drvStart || dispatchAddr >= drvEnd) { LogAlert(ALERT_DISPATCH_HOOK,
"SUSPICIOUS: %ws MajorFunction[%d] at 0x%llx is " "outside driver image [0x%llx -
0x%llx]", DriverName, MajorFunction, dispatchAddr, drvStart, drvEnd); } return
STATUS_SUCCESS; } // — Scan all keyboard-related drivers — NTSTATUS
ScanKeyboardDrivers(VOID) { // Find Kbclass driver object UNICODE_STRING kbclassName;
RtlInitUnicodeString(&kbclassName, L"\\Driver\\Kbclass"); PDRIVER_OBJECT kbclassDrv =
NULL; NTSTATUS status = ObReferenceObjectByName( &kbclassName, OBJ_CASE_INSENSITIVE,
NULL, 0, *IoDriverObjectType, KernelMode, NULL, (PVOID*)&kbclassDrv); if
(NT_SUCCESS(status)) { // Check IRP_MJ_READ dispatch (0x03)
CheckDispatchHook(kbclassDrv, IRP_MJ_READ, L"Kbclass"); // Also check
IRP_MJ_DEVICE_CONTROL (0x0E) CheckDispatchHook(kbclassDrv, IRP_MJ_DEVICE_CONTROL,
L"Kbclass"); ObDereferenceObject(kbclassDrv); } // Find Kbdhid driver object
UNICODE_STRING kbdhidName; RtlInitUnicodeString(&kbdhidName, L"\\Driver\\Kbdhid");
PDRIVER_OBJECT kbdhidDrv = NULL; status = ObReferenceObjectByName( &kbdhidName,
OBJ_CASE_INSENSITIVE, NULL, 0, *IoDriverObjectType, KernelMode, NULL,
(PVOID*)&kbdhidDrv); if (NT_SUCCESS(status)) { CheckDispatchHook(kbdhidDrv, IRP_MJ_READ,
L"Kbdhid"); CheckDispatchHook(kbdhidDrv, IRP_MJ_INTERNAL_DEVICE_CONTROL, L"Kbdhid");
ObDereferenceObject(kbdhidDrv); } return STATUS_SUCCESS; }
```

## 4.3 Kernel .text Section Integrity Verification

Inline patching of the kernel's .text section is a powerful rootkit technique that modifies the executable code of the kernel itself. This module periodically computes SHA-256 hashes of the .text sections of ntoskrnl.exe and all loaded drivers, comparing them against a secure baseline stored in a TPM-protected registry key (or, for systems without TPM, in a signed blob within the driver's own resources). The verification is performed by mapping the physical pages backing the .text section and reading them directly, avoiding any potential copy-on-write

or memory mapping tricks that a compromised kernel could use to hide modifications. To minimize performance impact, the verification is staggered across multiple DPC (Deferred Procedure Call) invocations, with each invocation verifying a small chunk of the .text section.

```
// ===== // Kernel .text Section
Integrity Verification // SHA-256 hash comparison against TPM-protected baseline //
IRQL: PASSIVE_LEVEL (uses CNG/BCrypt for hashing) //
===== #define HASH_CHUNK_SIZE (64
* 1024) // 64 KB per DPC #define MAX_BASELINE_ENTRIES 128 typedef struct _BASELINE_ENTRY
{ WCHAR ModuleName[64]; ULONG64 TextSectionBase; ULONG TextSectionSize; UCHAR
Sha256Hash[32]; // Expected hash BOOLEAN Verified; } BASELINE_ENTRY, *PBASELINE_ENTRY;
BASELINE_ENTRY g_Baseline[MAX_BASELINE_ENTRIES]; ULONG g_BaselineCount = 0; // —
Compute SHA-256 of a memory region — NTSTATUS ComputeSha256(PVOID Address, ULONG Size,
UCHAR HashOut[32]) { BCrypt_ALG_HANDLE hAlg = NULL; BCrypt_HASH_HANDLE hHash = NULL;
NTSTATUS status; status = BCryptOpenAlgorithmProvider( &hAlg, BCrypt_SHA256_ALGORITHM,
NULL, 0); if (!NT_SUCCESS(status)) return status; status = BCryptCreateHash(hAlg,
&hHash, NULL, 0, NULL, 0, 0); if (!NT_SUCCESS(status)) {
BCryptCloseAlgorithmProvider(hAlg, 0); return status; } // Hash in chunks to handle
large .text sections PCHAR ptr = (PCHAR)Address; ULONG remaining = Size; while
(remaining > 0) { ULONG chunk = min(remaining, HASH_CHUNK_SIZE); status =
BCryptHashData(hHash, ptr, chunk, 0); if (!NT_SUCCESS(status)) break; ptr += chunk;
remaining -= chunk; } if (NT_SUCCESS(status)) { status = BCryptFinishHash(hHash,
HashOut, 32, 0); } BCryptDestroyHash(hHash); BCryptCloseAlgorithmProvider(hAlg, 0);
return status; } // — Verify single module .text section — NTSTATUS
VerifyModuleIntegrity(PBASELINE_ENTRY Entry) { UCHAR currentHash[32]; NTSTATUS status =
ComputeSha256( (PVOID)Entry->TextSectionBase, Entry->TextSectionSize, currentHash); if
(!NT_SUCCESS(status)) return status; // Constant-time comparison to avoid timing
side-channel ULONG diff = 0; for (int i = 0; i < 32; i++) { diff |= currentHash[i] ^
Entry->Sha256Hash[i]; } if (diff != 0) { // MISMATCH: .text section has been modified
LogAlert(ALERT_TEXT_PATCH, "INTEGRITY VIOLATION: %ws .text section hash mismatch! "
"Base=0x%llx Size=0x%X. Possible inline patching.", Entry->ModuleName,
Entry->TextSectionBase, Entry->TextSectionSize); // Compute and log which region changed
// (granular comparison for forensic analysis) Entry->Verified = FALSE; return
STATUS_DATA_ERROR; } Entry->Verified = TRUE; return STATUS_SUCCESS; } // — Build
initial baseline from loaded modules — NTSTATUS BuildBaseline(VOID) { // Enumerate
loaded modules via // AuxKlibQueryModuleInformation (or direct PEB walking)
PRTL_PROCESS_MODULES modules = NULL; NTSTATUS status = AuxKlibQueryModuleInformation(
&modules, sizeof(RTL_PROCESS_MODULES), NULL); // For each module, find .text section and
compute hash for (ULONG i = 0; i < modules->NumberOfModules; i++) {
PRTL_PROCESS_MODULE_INFORMATION mod = &modules->Modules[i]; // Parse PE headers to find
.text section PCHAR base = (PCHAR)mod->ImageBase; PIMAGE_DOS_HEADER dos =
(PIMAGE_DOS_HEADER)base; PIMAGE_NT_HEADERS64 nt = (PIMAGE_NT_HEADERS64) (base +
dos->e_lfanew); PIMAGE_SECTION_HEADER section = IMAGE_FIRST_SECTION(nt); for (USHORT s =
0; s < nt->FileHeader.NumberOfSections; s++, section++) { if (memcmp(section->Name,
".text", 5) == 0) { PBASELINE_ENTRY entry = &g_Baseline[g_BaselineCount];
wcsncpy_s(entry->ModuleName, 64, mod->FullPathName, _TRUNCATE); entry->TextSectionBase =
(ULONG64)base + section->VirtualAddress; entry->TextSectionSize =
section->Misc.VirtualSize; // Compute and store hash ComputeSha256(
(PVOID)entry->TextSectionBase, entry->TextSectionSize, entry->Sha256Hash);
g_BaselineCount++; break; } } } // Store baseline in TPM-protected registry key //
(production: use Tbsip_Submit_Command for TPM seal) return STATUS_SUCCESS; }
```

## 5. Crypto-Agnostic Memory and Cache Mitigation Module

This is the core mitigation engine of the driver, designed to protect sensitive data from classical memory and cache side-channel attacks without attempting to identify specific cryptographic modules. Instead, it uses a tagging mechanism to mark "sensitive memory pages" containing keystroke buffers, key material, or crypto I/O buffers, and applies a suite of mitigations including KPTI-style page table isolation, VERW/L1D cache flushing on security boundaries, speculation control barriers, and optional cache partitioning via Intel CAT. The design is inspired by the mitigation strategies employed by modern operating systems (Linux KPTI, Windows Retpoline, VERW-based MDS mitigation) but applies them more aggressively and broadly, on the assumption that any data crossing a security boundary is potentially sensitive and worth protecting.

### 5.1 Sensitive Memory Identification and Isolation

The first step in the mitigation pipeline is identifying which memory pages contain sensitive data. Since the driver cannot reliably distinguish cryptographic code from other code (as stated in the design constraints), it employs a multi-pronged approach. First, it provides a custom allocation API (analogous to `ExAllocatePoolWithTag` but with an additional sensitivity flag) that callers use to allocate buffers that will contain keystroke data, key material, or crypto I/O. Pages allocated through this API are tagged in an internal bitmap and, where the PTE format allows, marked with a custom bit in the PTE's reserved fields. Second, the driver applies a conservative heuristic: any page that is mapped in both user and kernel address spaces (crossing the user/kernel boundary) or mapped in multiple process address spaces is treated as potentially sensitive. Third, the driver hooks the keyboard ISR and marks the pages containing the keyboard ring buffer as sensitive.

```
// ===== // Sensitive Memory
// Tagging & Isolation // Tags pages containing key material, keystroke buffers, etc. //
// IRQL: Varies (tagging at PASSIVE, flush at DISPATCH+) //
===== #define MAX_SENSITIVE_PAGES
65536 // — Sensitivity flags (stored in internal bitmap) — #define SENS_FLAG_KEYSTROKE
0x01 // Keyboard ring buffer #define SENS_FLAG_KEY_MATERIAL 0x02 // Crypto key buffer
#define SENS_FLAG_CRYPTIO_IO 0x04 // Crypto input/output #define SENS_FLAG_CROSS_BOUND
0x08 // User/kernel cross-bound #define SENS_FLAG_MULTI_MAP 0x10 // Mapped in multiple
ASIDs typedef struct _SENS_PAGE_ENTRY { ULONG64 PageFrameNumber; UCHAR SensitivityFlags;
UCHAR Pad[3]; ULONG OwningProcessId; // 0 = kernel-only } SENS_PAGE_ENTRY,
*PSENS_PAGE_ENTRY; SENS_PAGE_ENTRY g_SensPages[MAX_SENSITIVE_PAGES]; ULONG
g_SensPageCount = 0; KSPIN_LOCK g_SensLock; // — Custom allocation API for sensitive
buffers — PVOID SensAllocatePool( POOL_TYPE PoolType, SIZE_T Size, ULONG Tag, UCHAR
SensitivityFlags) { PVOID ptr = ExAllocatePoolWithTag(PoolType, Size, Tag); if (!ptr)
return NULL; // Tag all pages covered by this allocation ULONG64 startPfn = (ULONG64)ptr
>> PAGE_SHIFT; ULONG64 endPfn = ((ULONG64)ptr + Size + PAGE_SIZE - 1) >> PAGE_SHIFT;
KIRQL oldIrql; KeAcquireSpinLock(&g_SensLock, &oldIrql); for (ULONG64 pfn = startPfn;
pfn < endPfn; pfn++) { if (g_SensPageCount >= MAX_SENSITIVE_PAGES) break;
```

```

PSENS_PAGE_ENTRY entry = &g_SensPages[g_SensPageCount]; entry->PageFrameNumber = pfn;
entry->SensitivityFlags = SensitivityFlags; entry->OwningProcessId =
(ULONG)(ULONG_PTR)PsGetCurrentProcessId(); g_SensPageCount++; }
KeReleaseSpinLock(&g_SensLock, oldIrql); // If this is a keystroke buffer, immediately
apply // KPTI-style isolation: remove from user page tables if (SensitivityFlags &
(SENS_FLAG_KEYSTROKE | SENS_FLAG_KEY_MATERIAL)) { SensIsolateFromUserPageTables(ptr,
Size); } return ptr; } // — KPTI-style page table isolation — // Remove sensitive pages
from user-mode page tables NTSTATUS SensIsolateFromUserPageTables(PVOID Address, SIZE_T
Size) { // On Windows 11 with KPTI enabled, the user page tables // are a separate set of
PML4 tables used when running in // Ring 3. We need to clear the PTE entries for our //
sensitive pages in the user page tables only. // CR3 points to the current PML4. KPTI
maintains two // sets: one for kernel mode (full), one for user mode // (limited). We
modify the user-mode copy. ULONG64 cr3 = __readcr3(); ULONG64 pml4Base = cr3 & ~0xFFF;
PVOID pml4 = MmMapIoSpace( (PHYSICAL_ADDRESS){.QuadPart = (LONGLONG)pml4Base},
PAGE_SIZE, MmNonCached); if (!pml4) return STATUS_INSUFFICIENT_RESOURCES; ULONG64
startVirt = (ULONG64)Address; ULONG64 endVirt = startVirt + Size; for (ULONG64 virt =
startVirt; virt < endVirt; virt += PAGE_SIZE) { // Walk 4-level page table hierarchy: //
PML4[i] -> PDPT[j] -> PD[k] -> PT[l] -> PTE ULONG64 pml4Idx = (virt >> 39) & 0x1FFF;
ULONG64 pdptIdx = (virt >> 30) & 0x1FFF; ULONG64 pdIdx = (virt >> 21) & 0x1FFF; ULONG64
ptIdx = (virt >> 12) & 0x1FFF; // Walk PML4 -> PDPT -> PD -> PT -> PTE // Clear the PTE in
user page tables // (Detailed page table walking omitted for brevity; // production code
uses self-referencing PTE technique) // Mark the PTE as non-executable and
non-speculative // PTE.NX = 1, and if supported, set bit 63 // (reserved-for-software) as
"sensitive" marker } MmUnmapIoSpace(pml4, PAGE_SIZE); // Flush TLB for affected pages
__invlpg(Address); return STATUS_SUCCESS; } // — Conservative heuristic: cross-boundary
detection — // Scan working sets for pages mapped in both user & kernel VOID
SensDetectCrossBoundaryPages(VOID) { // For each process, walk its VAD tree and check //
if any kernel-mode pages are also mapped in user space // This catches cases where a
compromised kernel // mapped sensitive kernel pages into user space // (a prerequisite
for Meltdown-style attacks) PEPROCESS process = NULL; for (ULONG pid = 4; pid < 65536;
pid += 4) { NTSTATUS status = PsLookupProcessByProcessId( (HANDLE)(ULONG_PTR)pid,
&process); if (!NT_SUCCESS(status)) continue; // Walk VAD tree (EPROCESS->VadRoot) //
For each VAD, check if the underlying PFN // is in g_SensPages[] and if the VAD allows //
user-mode access (this would be a violation) ObDereferenceObject(process); } }

```

## 5.2 Security-Boundary Cache and Buffer Flushing

The core of the mitigation strategy is inserting appropriate buffer-clearing sequences at security boundaries. A "security boundary" is defined as any transition where the trust domain changes: a context switch to or from a process that owns sensitive pages, an entry into or exit from a sensitive kernel path (keyboard ISR, crypto syscall, disk encryption path), or a transition between a hyper-threaded core's sibling logical processors when one is running sensitive code. The exact flush sequence depends on the CPU's microarchitectural capabilities, determined by reading IA32\_ARCH\_CAPABILITIES and related MSRs at driver initialization. The following code implements the CPU capability probing and the cheapest-safe-mitigation selection logic, following the same decision tree used by the Linux kernel's MDS/L1TF mitigation code.

```

// ===== // Security-Boundary
Cache & Buffer Flushing // VERW + L1D flush on context switches & ISR entry/exit // IRQL:
DISPATCH_LEVEL or higher (no paged access) //
===== // — CPU Feature Flags
(from CPUID + MSRs) — #define CPU_FEAT_MDS_NO 0x0001 // IA32_ARCH_CAP MDS_NO #define
CPU_FEAT_L1TF_NO 0x0002 // IA32_ARCH_CAP L1TF_NO #define CPU_FEAT_RDCL_NO 0x0004 //
IA32_ARCH_CAP RDCL_NO #define CPU_FEAT_IBRS_ALL 0x0008 // IA32_ARCH_CAP IBRS_ALL #define
CPU_FEAT_VERW_FLUSH 0x0010 // VERW clears CPU buffers #define CPU_FEAT_L1D_FLUSH 0x0020

```

```

// IA32_FLUSH_CMD supported #define CPU_FEAT_SSB_NO 0x0040 // IA32_ARCH_CAP SSB_NO
#define CPU_FEAT_PSFD 0x0080 // PRED_CMD has PSFD #define CPU_FEAT_CAT_L3 0x0100 //
Cache Alloc Tech for L3 // — Mitigation Strategy Selection — typedef enum
_MITIGATION_STRATEGY { MitigateNone = 0, // CPU not vulnerable MitigateVerwOnly, // VERW
sufficient (MDS-only) MitigateLldFlushOnly, // L1D flush only (L1TF-only)
MitigateVerwPlusLld, // VERW + L1D flush (MDS + L1TF) MitigateFullSpectrum // All
mitigations (worst case) } MITIGATION_STRATEGY; typedef struct _CPU_MITIGATIONS { ULONG
Features; // CPU_FEAT_* bitmap MITIGATION_STRATEGY Strategy; // Selected strategy
ULONG64 VerwSelector; // Selector for VERW operand ULONG FlushCostNs; // Estimated cost
in nanoseconds } CPU_MITIGATIONS, *PCPU_MITIGATIONS; CPU_MITIGATIONS
g_CpuMit[MAXIMUM_PROCESSORS]; // — Probe CPU capabilities at driver init — NTSTATUS
ProbeCpuCapabilities(ULONG CpuIndex) { PCPU_MITIGATIONS mit = &g_CpuMit[CpuIndex];
mit->Features = 0; // Step 1: CPUID leaf 0x07 sub-leaf 0x00 int cpuid_07_00[4];
__cpuidex(cpuid_07_00, 7, 0); // IBRS/IBPB support (EDX bit 26) if (cpuid_07_00[3] & (1
<< 26)) mit->Features |= CPU_FEAT_IBRS_ALL; // Step 2: Read IA32_ARCH_CAPABILITIES (if
supported) // CPUID leaf 0x07 sub-leaf 0x00 EDX bit 29 if (cpuid_07_00[3] & (1 << 29)) {
ULONG64 archCap = __readmsr(0x10A); if (archCap & (1ULL << 0)) // RDCL_NO mit->Features
|= CPU_FEAT_RDCL_NO; if (archCap & (1ULL << 4)) // MDS_NO mit->Features |=
CPU_FEAT_MDS_NO; if (archCap & (1ULL << 5)) // L1TF_NO mit->Features |=
CPU_FEAT_L1TF_NO; if (archCap & (1ULL << 6)) // IBRS_ALL mit->Features |=
CPU_FEAT_IBRS_ALL; if (archCap & (1ULL << 7)) // SSB_NO mit->Features |=
CPU_FEAT_SSB_NO; } // Step 3: Check VERW MDS-clearing microcode // CPUID leaf 0x07
sub-leaf 0x00 EDX bit 10 (MD_CLEAR) if (cpuid_07_00[3] & (1 << 10)) mit->Features |=
CPU_FEAT_VERW_FLUSH; // Step 4: Check IA32_FLUSH_CMD support // CPUID leaf 0x07 sub-leaf
0x00 EDX bit 28 if (cpuid_07_00[3] & (1 << 28)) mit->Features |= CPU_FEAT_L1D_FLUSH; //
Step 5: Check CAT support // CPUID leaf 0x10 sub-leaf 0x00 EBX bit 1 int cpuid_10_00[4];
__cpuidex(cpuid_10_00, 0x10, 0); if (cpuid_10_00[1] & (1 << 1)) mit->Features |=
CPU_FEAT_CAT_L3; // Step 6: Select cheapest safe mitigation strategy BOOL mdsVuln =
!(mit->Features & CPU_FEAT_MDS_NO); BOOL l1tfVuln = !(mit->Features & CPU_FEAT_L1TF_NO);
if (!mdsVuln && !l1tfVuln) { mit->Strategy = MitigateNone; mit->FlushCostNs = 0; } else
if (mdsVuln && !l1tfVuln) { mit->Strategy = MitigateVerwOnly; mit->FlushCostNs = 50; //
~50ns for VERW } else if (!mdsVuln && l1tfVuln) { mit->Strategy = MitigateLldFlushOnly;
mit->FlushCostNs = 200; // ~200ns for L1D flush } else { // Both MDS and L1TF vulnerable
if (mit->Features & CPU_FEAT_VERW_FLUSH) { mit->Strategy = MitigateVerwPlusLld;
mit->FlushCostNs = 250; } else { mit->Strategy = MitigateFullSpectrum; mit->FlushCostNs
= 500; } } // Prepare VERW operand: a selector pointing to a // data segment descriptor
with the "cleared by VERW" // semantics. Linux uses the __kernel_ds selector. // On
Windows, we use a suitable data segment. mit->VerwSelector = 0x68; // Data segment
selector return STATUS_SUCCESS; } // — Execute mitigation flush at security boundary —
// Called on: context switch, ISR entry/exit, syscall boundary FORCEINLINE VOID
ExecuteSecurityBoundaryFlush(ULONG CpuIndex) { PCPU_MITIGATIONS mit =
&g_CpuMit[CpuIndex]; switch (mit->Strategy) { case MitigateNone: break; case
MitigateVerwOnly: // VERW instruction: on CPUs with MD_CLEAR microcode, // this clears
all microarchitectural buffers // (store buffer, load buffer, fill buffer, L1D)
__mm_mfence(); __asm { verw word ptr [mit->VerwSelector] } break; case
MitigateLldFlushOnly: // Write to IA32_FLUSH_CMD to flush L1D cache if (mit->Features &
CPU_FEAT_L1D_FLUSH) { __writemsr(0x10B, 1ULL); // L1D_FLUSH } break; case
MitigateVerwPlusLld: // Both VERW and L1D flush needed __mm_mfence(); __asm { verw word
ptr [mit->VerwSelector] } if (mit->Features & CPU_FEAT_L1D_FLUSH) { __writemsr(0x10B,
1ULL); // L1D_FLUSH } break; case MitigateFullSpectrum: // All available mitigations
__mm_mfence(); __asm { verw word ptr [mit->VerwSelector] } if (mit->Features &
CPU_FEAT_L1D_FLUSH) { __writemsr(0x10B, 1ULL); // L1D_FLUSH } // Serialize to prevent
speculative execution past // the flush point __mm_lfence(); break; } } // — Context
switch hook — // Installed via KeRegisterProcessorChangeCallback // or by patching
SwapContext (advanced technique) VOID SensContextSwitchHook( PKTHREAD OldThread,
PKTHREAD NewThread) { // Check if either the old or new thread's process // owns
sensitive pages HANDLE oldPid = PsGetProcessId( KeGetProcessFromThread(OldThread));
HANDLE newPid = PsGetProcessId( KeGetProcessFromThread(NewThread)); BOOLEAN oldSens =
IsProcessSensitive(oldPid); BOOLEAN newSens = IsProcessSensitive(newPid); if (oldSens ||
newSens) { // Security boundary transition detected
ExecuteSecurityBoundaryFlush(KeGetCurrentProcessorIndex()); } } // — Check if a process
owns sensitive pages — BOOLEAN IsProcessSensitive(HANDLE ProcessId) { ULONG pid =
(ULONG)(ULONG_PTR)ProcessId; KIRQL oldIrql; KeAcquireSpinLock(&g_SensLock, &oldIrql);

```

```

for (ULONG i = 0; i < g_SensPageCount; i++) { if (g_SensPages[i].OwningProcessId == pid)
{ KeReleaseSpinLock(&g_SensLock, oldIrql); return TRUE; } }
KeReleaseSpinLock(&g_SensLock, oldIrql); return FALSE; }

```

## 5.3 Speculation and Transient Execution Controls

Spectre variant 1 (bounds check bypass) and variant 2 (branch target injection) exploits can cause the CPU to transiently execute code paths that leak sensitive data through microarchitectural side channels. This module inserts speculation barriers (LFENCE, MFENCE) at critical points in sensitive kernel paths and enables hardware speculation controls (IBRS, STIBP) where supported. The barriers are placed before any access to tagged (sensitive) memory pages in the keyboard ISR, crypto-related syscalls, and disk encryption paths. Additionally, the module validates all bounds checks on user pointers before they are used in sensitive paths, following the pattern of Linux's `array_index_nospec()` macro that masks array indices to prevent speculative out-of-bounds access.

```

// ===== // Speculation Control
for Sensitive Kernel Paths // Barriers + IBRS/STIBP + bounds hardening // IRQL: Varies
(barriers are IRQL-agnostic) //
===== // — Speculation barrier
macros — #define SPECULATION_BARRIER() _mm_lfence() #define MEMORY_BARRIER()
_mm_mfence() // — Safe array index (prevents Spectre v1 gadget) — // Masks the index to
prevent speculative out-of-bounds FORCEINLINE ULONG SafeArrayIndex(ULONG Index, ULONG
ArraySize) { // Constant-time mask: if Index >= ArraySize, mask = 0 ULONG mask =
-(ULONG)(Index < ArraySize); // 0 or 0xFFFFFFFF return Index & mask; } // — Safe user
pointer dereference — // Validates + serializes before accessing tagged memory
FORCEINLINE NTSTATUS SafeAccessUserPointer( PVOID UserPtr, PVOID KernelBuf, ULONG Size,
BOOLEAN IsSensitivePath) { // Step 1: Validate user pointer is in user-mode range if
((ULONG64)UserPtr > MmHighestUserAddress) { return STATUS_ACCESS_VIOLATION; } // Step 2:
If accessing from a sensitive path, // insert speculation barrier BEFORE the access if
(IsSensitivePath) { SPECULATION_BARRIER(); } // Step 3: Probe and lock user buffer __try
{ ProbeForRead(UserPtr, Size, sizeof(CHAR)); RtlCopyMemory(KernelBuf, UserPtr, Size); }
__except(EXCEPTION_EXECUTE_HANDLER) { return STATUS_ACCESS_VIOLATION; } return
STATUS_SUCCESS; } // — Enable IBRS/STIBP for sensitive paths — VOID
EnableSpeculationControls(ULONG CpuIndex) { PCPU_MITIGATIONS mit = &g_CpuMit[CpuIndex];
if (!(mit->Features & CPU_FEAT_IBRS_ALL)) { // CPU does not support IBRS, use retpoline
// (Retpoline is a software mitigation applied at // compile time via /Qspectre on MSVC)
return; } // Enable IBRS (Indirect Branch Restricted Speculation) ULONG64 specCtrl =
__readmsr(0x48); // IA32_SPEC_CTRL specCtrl |= (1ULL << 0); // IBRS specCtrl |= (1ULL <<
1); // STIBP (Single Thread Indirect // Branch Predictor) __writemsr(0x48, specCtrl); }
// — Disable IBRS after sensitive path (performance) — VOID
DisableSpeculationControls(ULONG CpuIndex) { PCPU_MITIGATIONS mit = &g_CpuMit[CpuIndex];
if (!(mit->Features & CPU_FEAT_IBRS_ALL)) return; ULONG64 specCtrl = __readmsr(0x48);
specCtrl &= ~(1ULL << 0); // Disable IBRS // Keep STIBP enabled if sibling core may run
untrusted // (based on scheduling policy) __writemsr(0x48, specCtrl); // Issue IBPB
(Indirect Branch Prediction Barrier) // to prevent cross-process branch target injection
__writemsr(0x49, 1ULL); // IA32_PRED_CMD.IBPB } // — Keyboard ISR hardened entry — //
Called at HIGH_LEVEL when keyboard interrupt fires VOID HardenedKeyboardIsr(PKINTERRUPT
Interrupt, PVOID Context) { // Step 1: Speculation barrier at ISR entry
SPECULATION_BARRIER(); // Step 2: Flush microarchitectural buffers // (prevents MDS/L1TF
leakage of keystroke data // from previous context)
ExecuteSecurityBoundaryFlush(KeGetCurrentProcessorIndex()); // Step 3: Read keyboard
data from I/O port 0x60 UCHAR scanCode = __inbyte(0x60); // Step 4: Store in
sensitive-tagged ring buffer // (this page is tagged SENS_FLAG_KEYSTROKE)
PSENS_PAGE_ENTRY sensEntry = FindSensPageByVirtAddr( g_KbdRingBuffer); if (sensEntry) {
// Page is tagged; speculation barriers already in place ULONG idx =

```

```
SafeArrayIndex(g_KbdWriteIdx, KBD_RING_SIZE); g_KbdRingBuffer[idx] = scanCode;
g_KbdWriteIdx = (g_KbdWriteIdx + 1) % KBD_RING_SIZE; } // Step 5: Flush again before
leaving sensitive path // (prevents leakage to next context on same core)
ExecuteSecurityBoundaryFlush(KeGetCurrentProcessorIndex()); // Step 6: Speculation
barrier at ISR exit SPECULATION_BARRIER(); }
```

## 5.4 Cache Partitioning via CAT (Cache Allocation Technology)

Intel's Cache Allocation Technology (CAT) allows partitioning of the last-level cache (L3) by assigning different "classes of service" (CLOS) to different workloads, each with a defined capacity bit mask (CBM) that determines which cache ways the workload may occupy. This module uses CAT to reserve a small portion of the L3 cache exclusively for sensitive code and data (keyboard ISR, crypto buffers), ensuring that untrusted Ring 3 processes cannot evict sensitive data from the LLC. The CAT configuration is done via the MSR\_PQR\_ASSOC register (which assigns the current logical processor to a CLOS) and the MSR\_L3\_CBMBASE + CLOS \* MSR\_L3\_CBMSIZE registers (which define the CBM for each CLOS). The following code implements CAT initialization, CBM configuration, and CLOS assignment for the sensitive and non-sensitive partitions.

```
// ===== // Cache Allocation
Technology (CAT) Configuration // Partitions L3 cache for sensitive vs. non-sensitive
data // IRQ: PASSIVE_LEVEL (init), DISPATCH_LEVEL (assignment) //
===== // — CAT MSR Addresses —
#define MSR_PQR_ASSOC 0xC8F #define MSR_L3_CBMBASE 0xC90 // + CLOS * stride #define
MSR_L3_QOS_CFG 0xC81 // CAT enable / CDP mode // — CAT Configuration — typedef struct
_CatConfig { BOOLEAN Enabled; ULONG TotalCacheWays; // e.g., 11 or 20 ULONG
SensitiveWays; // Ways reserved for sensitive ULONG SensitiveClos; // CLOS ID for
sensitive (e.g., 1) ULONG DefaultClos; // CLOS ID for default (e.g., 0) ULONG64
SensitiveCbm; // CBM for sensitive partition ULONG64 DefaultCbm; // CBM for default
partition ULONG CbmStride; // MSR stride between CLOS entries } CAT_CONFIG,
*PCAT_CONFIG; CAT_CONFIG g_CatConfig = {0}; // — Initialize CAT partitioning —
NTSTATUS CatInitialize(ULONG CpuIndex) { PCPU_MITIGATIONS mit = &g_CpuMit[CpuIndex]; if
(! (mit->Features & CPU_FEAT_CAT_L3)) { g_CatConfig.Enabled = FALSE; return
STATUS_NOT_SUPPORTED; } // Step 1: Query CAT capabilities via CPUID leaf 0x10 int
cpuid_10_01[4]; // Sub-leaf 1 = L3 CAT __cpuidex(cpuid_10_01, 0x10, 1); // Bit [4:0] of
EAX = length of CBM (number of ways - 1) ULONG cbmLen = cpuid_10_01[0] & 0x1F;
g_CatConfig.TotalCacheWays = cbmLen + 1; // EBX bit [11:0] = which cache ways can be used
( bitmask) ULONG availableWays = cpuid_10_01[1] & 0xFFFF; // EDX [15:0] = max CLOS IDs
supported ULONG maxClos = cpuid_10_01[3] & 0xFFFF; if (maxClos < 2) { g_CatConfig.Enabled
= FALSE; return STATUS_NOT_SUPPORTED; } // Step 2: Define partition strategy // Reserve
20-30% of L3 cache for sensitive data // Example: with 11 ways, reserve 3 ways for
sensitive ULONG sensitiveWays = max(2, g_CatConfig.TotalCacheWays / 4);
g_CatConfig.SensitiveWays = sensitiveWays; g_CatConfig.SensitiveClos = 1;
g_CatConfig.DefaultClos = 0; g_CatConfig.CbmStride = 1; // Typically 1 MSR per CLOS //
Step 3: Build CBMs (capacity bit masks) // Sensitive CBM: highest-order bits (e.g., ways
8-10) g_CatConfig.SensitiveCbm = 0; for (ULONG i = 0; i < sensitiveWays; i++) {
g_CatConfig.SensitiveCbm |= (1ULL << (cbmLen - i)); } // Default CBM: remaining ways
(e.g., ways 0-7) g_CatConfig.DefaultCbm = ((1ULL << (cbmLen + 1)) - 1) &
~g_CatConfig.SensitiveCbm; // Step 4: Write CBMs to MSRs // CLOS 0 (default): all
non-sensitive ways __writemsr(MSR_L3_CBMBASE + 0 * g_CatConfig.CbmStride,
g_CatConfig.DefaultCbm); // CLOS 1 (sensitive): reserved ways __writemsr(MSR_L3_CBMBASE
+ 1 * g_CatConfig.CbmStride, g_CatConfig.SensitiveCbm); // Step 5: Enable CAT (set CLOS
enable bit) ULONG64 qosCfg = __readmsr(MSR_L3_QOS_CFG); qosCfg |= (1ULL << 0); // Enable
```

```

CAT __writemsr(MSR_L3_QOS_CFG, qosCfg); g_CatConfig.Enabled = TRUE; return
STATUS_SUCCESS; } // — Assign CLOS to current logical processor — // Called when
entering/leaving a sensitive code path FORCEINLINE VOID CatAssignClos(ULONG ClosId) { if
(!g_CatConfig.Enabled) return; // Write CLOS ID to PQR_ASSOC MSR // Bits [31:0] = CLOS ID
(used by hardware) // Bits [63:32] = RMID (for QOS monitoring, unused here) ULONG64 pqr =
(ULONG64)ClosId & 0xFFFFFFFF; __writemsr(MSR_PQR_ASSOC, pqr); } // — Usage in sensitive
kernel path — // Before entering keyboard ISR or crypto code: VOID
EnterSensitivePath(VOID) { CatAssignClos(g_CatConfig.SensitiveClos); } // After leaving
sensitive path: VOID LeaveSensitivePath(VOID) { CatAssignClos(g_CatConfig.DefaultClos);
}

```

## 5.5 Hyper-Threading and Cross-VM Considerations

Simultaneous multithreading (SMT, also known as Hyper-Threading on Intel CPUs) creates a side-channel risk because sibling logical processors share the L1 data cache, L1 instruction cache, and microarchitectural buffers such as the store buffer and load buffer. This means that an attacker running on one logical processor can potentially observe the memory access patterns of a victim running on the sibling logical processor, leaking sensitive data such as cryptographic keys or keystroke timing information. This module implements two complementary mitigations: a scheduling mitigation that avoids scheduling untrusted threads on the sibling core when a thread with tagged (sensitive) memory is running, and a buffer-flushing mitigation that applies VERW or L1D flush on transitions between security domains on the same core. The scheduling mitigation is implemented by setting processor affinity constraints via `KeSetSystemAffinityThreadEx`, while the flush mitigation uses the `ExecuteSecurityBoundaryFlush` function described in the previous section.

```

// ===== // SMT / Hyper-Threading
Mitigation // Sibling-core isolation for sensitive processes // IRQL: PASSIVE_LEVEL
(scheduling), DISPATCH_LEVEL (flush) //
===== // — SMT topology cache —
typedef struct _SMT_SIBLING { ULONG LogicalProcessor; ULONG SiblingProcessor; // Same
core, other HT BOOLEAN IsSensitiveThreadRunning; } SMT_SIBLING, *PSMT_SIBLING;
SMT_SIBLING g_SmtMap[MAXIMUM_PROCESSORS]; ULONG g_NumLogicalProcs = 0; // — Build SMT
topology at driver init — NTSTATUS SmtBuildTopology(VOID) { // Use CPUID leaf 0x0B
(Extended Topology) // to determine which logical processors share a core for (ULONG cpu
= 0; cpu < KeQueryMaximumProcessorCount(); cpu++) { int cpuid_0b[4]; __cpuidex(cpuid_0b,
0x0B, 1); // EDX bits [31:0] = x2APIC ID // Siblings share the same core ID (bits above
SMT level) ULONG apicId = cpuid_0b[3]; ULONG smtMask = (1 << (cpuid_0b[0] & 0x1F)) - 1;
ULONG coreId = apicId & ~smtMask; g_SmtMap[cpu].LogicalProcessor = cpu;
g_SmtMap[cpu].SiblingProcessor = MAXULONG; // Unknown yet // Find sibling with same
coreId but different smtId for (ULONG j = 0; j < cpu; j++) { int prevCpuid[4];
__cpuidex(prevCpuid, 0x0B, 1); ULONG prevApic = prevCpuid[3]; ULONG prevCore = prevApic
& ~smtMask; if (prevCore == coreId && prevApic != apicId) {
g_SmtMap[cpu].SiblingProcessor = j; g_SmtMap[j].SiblingProcessor = cpu; } } }
g_NumLogicalProcs = KeQueryMaximumProcessorCount(); return STATUS_SUCCESS; } // —
Schedule mitigation: avoid sibling core for // untrusted threads when sensitive thread is
running — NTSTATUS SmtIsolateSensitiveProcess(HANDLE ProcessId) { // Find which logical
processor the sensitive process // is currently running on ULONG currentCpu =
KeGetCurrentProcessorIndex(); // Mark this CPU as running a sensitive thread
g_SmtMap[currentCpu].IsSensitiveThreadRunning = TRUE; // Find sibling CPU ULONG
siblingCpu = g_SmtMap[currentCpu].SiblingProcessor; if (siblingCpu == MAXULONG) { // No
sibling (non-SMT system or last core) return STATUS_SUCCESS; } // If we can, IPI the
sibling core to flush its buffers // and delay scheduling untrusted threads there // (On
Windows, we use KeGenericCallDpc to execute // a DPC on the sibling processor)

```

```

KeGenericCallDpc(SmtSiblingFlushDpc, (PVOID)(ULONG_PTR)siblingCpu); return
STATUS_SUCCESS; } // — DPC to flush buffers on sibling processor — VOID
SmtSiblingFlushDpc( PKDPC Dpc, PVOID Context, PVOID SystemArgument1, PVOID
SystemArgument2) { ULONG cpu = (ULONG)(ULONG_PTR)Context; if
(KeGetCurrentProcessorIndex() == cpu) { // We are on the sibling core — flush buffers
ExecuteSecurityBoundaryFlush(cpu); // Optionally: request scheduler to avoid this core
// for untrusted processes via affinity hints } } // — Clear sensitive flag on context
switch out — VOID SmtClearSensitiveFlag(ULONG CpuIndex) {
g_SmtMap[CpuIndex].IsSensitiveThreadRunning = FALSE; // Also flush buffers when leaving
sensitive context ExecuteSecurityBoundaryFlush(CpuIndex); }

```

## 6. Secure Ring 0 to Ring 3 Communication

Since the kernel may be compromised, standard DeviceIoControl/ioctl communication channels could be intercepted by a rootkit that has hooked the driver dispatch routines or installed a filter device on top of our device object. This module implements two alternative secure communication mechanisms. The primary mechanism is a shared memory section with cryptographic authentication: the driver allocates a non-paged memory section that is mapped into both kernel and user address spaces, and each notification payload is authenticated with an HMAC computed using a key derived from a TPM-sealed secret. The secondary mechanism is a covert channel using an unused MSR (Model-Specific Register) as a signaling mechanism: the driver writes a specific value to the MSR, and the user-mode application reads it via a legitimate API, bypassing any device stack hooks. Both mechanisms are designed to resist detection and interception by a compromised kernel.

```

// ===== // Secure Ring 0 -> Ring
3 Communication // Method 1: HMAC-authenticated shared memory // Method 2: MSR-based
covert channel // IRQL: PASSIVE_LEVEL (shared mem), any (MSR) //
===== #define SHARED_MEM_SIZE
PAGE_SIZE #define HMAC_KEY_SIZE 32 #define NOTIFICATION_SLOTS 16 // — Notification
payload — typedef struct _SECURE_NOTIFICATION { ULONG Magic; // 0xDEADCODE ULONG
SequenceNumber; // Monotonically increasing ULONG AlertType; // ALERT_* constant ULONG
AlertLevel; // 0=info, 1=watch, 2=critical ULONG64 Timestamp; //
KeQueryPerformanceCounter ULONG64 Param1; // Alert-specific data ULONG64 Param2; //
Alert-specific data UCHAR Hmac[32]; // HMAC-SHA256 of above fields }
SECURE_NOTIFICATION, *PSECURE_NOTIFICATION; // — Shared memory region layout — typedef
struct _SHARED_MEM_REGION { ULONG WriteIndex; // Driver writes here ULONG ReadIndex; //
User-mode reads here ULONG DriverNonce; // Rolling nonce for HMAC ULONG Reserved;
SECURE_NOTIFICATION Notifications[NOTIFICATION_SLOTS]; } SHARED_MEM_REGION,
*PSHARED_MEM_REGION; PSHARED_MEM_REGION g_SharedMemKernel = NULL; PVOID
g_SharedMemUserMapping = NULL; UCHAR g_HmacKey[HMAC_KEY_SIZE]; // — Initialize shared
memory channel — NTSTATUS SecureCommInitialize(VOID) { // Allocate non-paged shared
memory PHYSICAL_ADDRESS highestAcceptable; highestAcceptable.QuadPart = -1;
g_SharedMemKernel = (PSHARED_MEM_REGION) MmAllocateContiguousMemory(SHARED_MEM_SIZE,
highestAcceptable); if (!g_SharedMemKernel) return STATUS_INSUFFICIENT_RESOURCES;
RtlZeroMemory(g_SharedMemKernel, SHARED_MEM_SIZE); // Create MDL for user-mode mapping
PMDL mdl = IoAllocateMdl(g_SharedMemKernel, SHARED_MEM_SIZE, FALSE, FALSE, NULL); if
(!mdl) return STATUS_INSUFFICIENT_RESOURCES; MmBuildMdlForNonPagedPool(mdl); // Derive
HMAC key from TPM-sealed secret // (Production: use TBS TPM2_Unseal on a pre-sealed blob)
// For now, generate a random key at boot ULONG64 seed = __rdtsc(); for (int i = 0; i <
HMAC_KEY_SIZE; i++) { g_HmacKey[i] = (UCHAR)((seed >> (i % 64)) & 0xFF); seed ^=
__rdtsc(); } return STATUS_SUCCESS; } // — Send secure notification — NTSTATUS

```

```
SecureCommNotify( ULONG AlertType, ULONG AlertLevel, ULONG64 Param1, ULONG64 Param2) {
    PSHARED_MEM_REGION shm = g_SharedMemKernel; ULONG idx = shm->WriteIndex %
    NOTIFICATION_SLOTS; PSECURE_NOTIFICATION notif = &shm->Notifications[idx]; notif->Magic
    = 0xDEADCODE; notif->SequenceNumber = InterlockedIncrement( (PLONG)&shm->DriverNonce);
    notif->AlertType = AlertType; notif->AlertLevel = AlertLevel; notif->Timestamp =
    KeQueryPerformanceCounter(NULL).QuadPart; notif->Param1 = Param1; notif->Param2 =
    Param2; // Compute HMAC-SHA256 over the notification // (excluding the Hmac field itself)
    BCryptAlg_HANDLE hAlg = NULL; BCrypt_HASH_HANDLE hHash = NULL; NTSTATUS status =
    BCryptOpenAlgorithmProvider( &hAlg, BCrypt_SHA256_ALGORITHM, NULL,
    BCrypt_ALG_HANDLE_HMAC_FLAG); if (!NT_SUCCESS(status)) return status; status =
    BCryptCreateHash(hAlg, &hHash, NULL, 0, g_HmacKey, HMAC_KEY_SIZE, 0); if
    (NT_SUCCESS(status)) { // Hash everything except the Hmac field ULONG dataLen =
    offsetof(SECURE_NOTIFICATION, Hmac); status = BCryptHashData(hHash, (PUCHAR)notif,
    dataLen, 0); if (NT_SUCCESS(status)) { BCryptFinishHash(hHash, notif->Hmac, 32, 0); }
    BCryptDestroyHash(hHash); } BCryptCloseAlgorithmProvider(hAlg, 0); // Advance write
    index (release store) InterlockedIncrement((PLONG)&shm->WriteIndex); return
    STATUS_SUCCESS; } // — Method 2: MSR Covert Channel — // Uses an unused MSR as a
    side-band signaling mechanism // The user-mode app reads this MSR via a legitimate //
    kernel interface (e.g., NtQuerySystemInformation // with a custom class, or a signed
    driver helper) #define COVERT_MSR_SIGNAL 0x00000150 // MSR_MISC_FEATURES // (or other
    unused MSR) #define SIGNAL_ALERT 0xCAFEBAFE #define SIGNAL_CLEAR 0x00000000 VOID
    SecureCommMsrSignal(ULONG AlertType) { // Write alert type to unused MSR bits // This is
    a fire-and-forget signal; user-mode polls ULONG64 value = ((ULONG64)AlertType << 32) |
    SIGNAL_ALERT; __writemsr(COVERT_MSR_SIGNAL, value); } VOID SecureCommMsrClear(VOID) {
    __writemsr(COVERT_MSR_SIGNAL, SIGNAL_CLEAR); }
```

## 7. Integration and Cross-Module Interactions

The five modules described in the previous sections are not independent silos; they form an integrated defense system where findings from one module can trigger enhanced actions in other modules. This section describes the cross-module interaction protocol, the shared state structure, and the overall event flow from detection to mitigation. The shared state is protected by a combination of spin locks (for DISPATCH\_LEVEL access) and a SHA-256 integrity hash that is verified periodically. If the integrity hash is found to be invalid, the driver enters a fail-safe mode where it applies maximum mitigations (full VERW + L1D flush + IBRS on all security boundaries) and signals the user-mode monitoring application with a critical alert.

### 7.1 Shared Driver State Structure

```
// ===== // Shared Driver State
// (Cross-Module Communication) // Integrity-verified, spin-lock protected //
// ===== typedef struct
_DRIVER_SHARED_STATE { // — Module 1: PMU Detection — volatile LONG PmuAlertLevel; //
0-2 volatile LONG PmuL1DMissRate; // Events/second volatile LONG PmuL2MissRate; volatile
LONG PmuRdtscFlaggedPid; // Flagged process // — Module 2: Hardware Keylogger —
volatile LONG HwDiscrepancyCount; // OS vs HW mismatch volatile LONG
HwDmaViolationCount; volatile LONG HwLastAlertCode; // — Module 3: Kernel Integrity —
volatile LONG IdtHookDetected; // Boolean volatile LONG DispatchHookDetected; volatile
LONG TextPatchDetected; ULONG FailedModuleHashes; // — Module 4: Mitigation Engine —
volatile LONG SensPageCount; volatile LONG ActiveMitigations; // Bitmask volatile LONG
```

```

FlushCount; // Total flushes volatile LONG FlushCostNs; // Running average ULONG
CurrentClos; // CAT CLOS assignment // — Module 5: Secure Comms — volatile LONG
NotificationsSent; volatile LONG MsrSignalsSent; // — Integrity Verification — UCHAR
StateHash[32]; // SHA-256 of above KSPIN_LOCK StateLock; BOOLEAN IntegrityValid; }
DRIVER_SHARED_STATE, *PDRIVER_SHARED_STATE; DRIVER_SHARED_STATE g_SharedState = {0}; //
— Verify shared state integrity — BOOLEAN VerifySharedStateIntegrity(VOID) { UCHAR
computedHash[32]; // Hash everything except the StateHash field itself ULONG offset =
offsetof(DRIVER_SHARED_STATE, StateHash); ComputeSha256((PUCHAR)&g_SharedState, offset,
computedHash); ULONG diff = 0; for (int i = 0; i < 32; i++) { diff |= computedHash[i] ^
g_SharedState.StateHash[i]; } g_SharedState.IntegrityValid = (diff == 0); return
g_SharedState.IntegrityValid; }

```

## 7.2 Cross-Module Event Flow

The cross-module event flow follows a publish-subscribe pattern. When a module detects an anomaly, it updates the shared state and calls the central event dispatcher. The event dispatcher then evaluates the overall threat level based on the combined state of all modules and triggers appropriate mitigation actions. For example, if Module 1 (PMU) detects a high cache miss rate, the dispatcher may increase the sensitivity of Module 4's flush thresholds and schedule more frequent integrity checks via Module 3. If Module 3 detects an IDT hook, the dispatcher may trigger an immediate L1D flush and lock down DMA access via Module 2's IOMMU controls. The following table summarizes the key cross-module interactions and their triggers.

| Trigger Module  | Event                       | Response Module | Action                                         |
|-----------------|-----------------------------|-----------------|------------------------------------------------|
| M1 (PMU)        | L1D miss rate > threshold   | M4 (Mitigation) | Immediate L1D flush + increase flush frequency |
| M1 (PMU)        | RDTSC rate > threshold      | M4 (Mitigation) | Enable IBRS/STIBP on affected core             |
| M2 (Hardware)   | DMA violation detected      | M4 (Mitigation) | Lock IOMMU context entry + flush all buffers   |
| M2 (Hardware)   | Filter driver discrepancy   | M3 (Integrity)  | Immediate dispatch routine scan                |
| M3 (Integrity)  | IDT hook detected           | M4 (Mitigation) | Full-spectrum flush + isolate affected core    |
| M3 (Integrity)  | .text hash mismatch         | M5 (Comms)      | Critical alert to user-mode + enter fail-safe  |
| M4 (Mitigation) | Sensitive process scheduled | M4 (CAT/SMT)    | Assign sensitive CLOS + isolate sibling        |
| M4 (Mitigation) | Shared state integrity fail | M4 (Mitigation) | Enter fail-safe: maximum mitigations           |

**Table 2.** Cross-module event flow: triggers and corresponding mitigation actions.

## 8. Constraints, IRQL Rules, and Best Practices

This section documents the critical constraints and best practices that must be followed when implementing and deploying this driver. Violation of these rules can lead to system crashes, data corruption, or, worse, creation of new attack surfaces that the driver was designed to prevent.

### 8.1 IRQL and Paging Constraints

The Windows IRQL (Interrupt Request Level) system imposes strict rules on what operations are legal at each level. The PMI handler runs at `HIGH_LEVEL` (IRQL 26 on x64), which means it cannot access any paged memory, acquire any spin locks that might cause a page fault, or call any kernel APIs that touch paged code or data. The IDT verification code runs at `DISPATCH_LEVEL` when invoked from a DPC, and at `PASSIVE_LEVEL` when invoked from the periodic integrity worker thread. The hardware bus walking code runs at `PASSIVE_LEVEL` because it uses `MmMapIoSpace`, which requires a non-arbitrary thread context. The mitigation flush functions (VERW, L1D flush) are safe to call at any IRQL because they only execute CPU instructions (no memory allocation, no API calls). The following table summarizes the IRQL constraints for each module.

| Module / Function       | IRQL                | Page d Access | Key Constraints                                                  |
|-------------------------|---------------------|---------------|------------------------------------------------------------------|
| PMI Handler (M1)        | HIGH_LEVEL (26)     | NO            | No paged memory, no kernel APIs, no spin locks that page fault   |
| PMU Configuration (M1)  | PASSIVE_LEVEL       | YES           | Uses <code>MmMapIoSpace</code> , <code>IoConnectInterrupt</code> |
| PCIe Bus Walking (M2)   | PASSIVE_LEVEL       | YES           | <code>MmMapIoSpace</code> requires non-arbitrary thread context  |
| IOMMU Verification (M2) | PASSIVE_LEVEL       | YES           | ACPI table parsing, physical memory mapping                      |
| IDT Verification (M3)   | DISPATCH or PASSIVE | Conditional   | SIDT safe at any IRQL; <code>MmMapIoSpace</code> at PASSIVE only |
| Dispatch Hook Scan (M3) | PASSIVE_LEVEL       | YES           | Object manager access requires PASSIVE                           |
| .text Hash Verify (M3)  | PASSIVE_LEVEL       | YES           | BCrypt API requires PASSIVE_LEVEL                                |
| VERW / L1D Flush (M4)   | ANY                 | NO            | Pure CPU instructions, no API calls, safe at HIGH_LEVEL          |

|                        |                |     |                                                  |
|------------------------|----------------|-----|--------------------------------------------------|
| CAT Configuration (M4) | PASSIVE_LEVEL  | YES | __writemsr is IRQL-agnostic but init needs CPUID |
| SMT Isolation (M4)     | DISPATCH_LEVEL | NO  | KeGenericCallDpc runs at DISPATCH                |
| Secure Comms (M5)      | PASSIVE_LEVEL  | YES | BCrypt HMAC, MDL mapping                         |
| MSR Covert Signal (M5) | ANY            | NO  | __writemsr is IRQL-agnostic                      |

**Table 3.** IRQL and paging constraints for each module function.

## 8.2 MSR and CPUID Access Best Practices

All MSR reads and writes in this driver use the compiler intrinsics `__readmsr()` and `__writemsr()`, which compile directly to the RDMSR and WRMSR instructions without any indirection through kernel APIs that could be hooked. Similarly, CPUID queries use `__cpuidex()` which compiles to the CPUID instruction. These intrinsics are safe at any IRQL and cannot be intercepted by software. However, the following caveats apply: WRMSR to IA32\_SPEC\_CTRL and IA32\_PRED\_CMD may be slow (10-50 microseconds depending on microcode), so they should not be called on hot paths without careful performance analysis. The IA32\_FLUSH\_CMD.L1D\_FLUSH operation is also expensive (typically 100-200 cycles), but this cost is acceptable when compared to the security risk of leaving L1D cache populated with sensitive data during a context switch. When reading IA32\_ARCH\_CAPABILITIES, the driver must first verify that the CPU supports this MSR by checking CPUID leaf 0x07 sub-leaf 0x00 EDX bit 29; reading an unsupported MSR causes a general protection fault.

## 8.3 Avoiding New Exploitable Gadgets

A critical concern when implementing security software is the risk of introducing new Spectre or MELTDOWN gadgets in the mitigation code itself. This driver follows several principles to minimize this risk. First, all bounds checks on user pointers in sensitive paths are followed by a speculation barrier (LFENCE) before the pointer is dereferenced, following the pattern of Linux's `array_index_nospec()` macro. Second, the driver never speculatively dereferences user-controlled pointers in paths that access tagged (sensitive) memory; instead, it copies user data into a kernel buffer first, validates it, and then accesses the tagged memory. Third, all indirect branches in the driver are compiled with `retpoline` (via `/Qspectre` on MSVC or `-mretpoline` on GCC/Clang) to prevent Spectre variant 2 exploitation. Fourth, the driver's own code is included in the Module 3 baseline hash verification, so any runtime modification of its `.text` section will be detected and flagged.

## 8.4 Performance Impact Assessment

The performance impact of the driver depends on the mitigation strategy selected by the CPU capability probing logic. On CPUs that are not vulnerable to MDS or L1TF (i.e., `IA32_ARCH_CAPABILITIES` reports `MDS_NO` and `L1TF_NO`), the driver's runtime overhead is minimal: only the periodic PMU counter reads and integrity hash computations, which are scheduled on DPCs and consume less than 0.1% of a single core. On CPUs that require VERW-based MDS mitigation, each security-boundary flush adds approximately 50 nanoseconds to the context switch time. On CPUs that require both VERW and L1D flush, the cost increases to approximately 250 nanoseconds per flush. Given that a typical context switch on Windows takes 5-10 microseconds, this represents a 2.5-5% overhead in the worst case. The CAT partitioning strategy has a one-time configuration cost at boot but no ongoing runtime cost. The SMT isolation mitigation may reduce overall system throughput by 5-15% when sensitive processes are running, as one logical processor per core is effectively reserved. However, this is an acceptable trade-off for systems that process highly sensitive data such as cryptographic keys or classified keystrokes.